VIT® Chennai

Vellore Institute of Technology, Chennai Campus

School of Computer Science and Engineering (SCOPE)

MINI PROJECT REPORT

A GIS-Based Multi-Scenario Routing and Evacuation Planner

An app-based project for the Design and Analysis of Algorithms course

Submitted by

Maanya Agrawal

Reg. No. 24BCE5052

B.Tech Computer Science and Engineering

Course: [Course name and code]

Faculty: [Faculty name]

October 2026

Abstract

This project presents **A GIS-Based Multi-Scenario Routing and Evacuation Planner**, a web application that runs on the real road network of Tambaram, Chennai. The road network (14,022 intersections and 15,402 road segments), 134 public places, 11,909 building locations and an elevation grid are downloaded from OpenStreetMap and SRTM and converted into a weighted graph.

Four scenarios (everyday traffic, flood, fire and earthquake) modify the graph by blocking roads, slowing roads or setting arrival deadlines, and every algorithm then runs unchanged on the modified graph. The application provides GPS-style routing with a selectable algorithm (Dijkstra, A*, bidirectional Dijkstra and Bellman-Ford) and k alternative routes (Yen), service-area analysis with multi-source Dijkstra, an evacuation planner built on min-cost max-flow with k-means zoning and greedy shelter placement, network vulnerability analysis (Tarjan bridges and Brandes betweenness), Monte Carlo earthquake reliability, and multi-stop trip planning (nearest neighbour, 2-opt and Held-Karp).

All algorithms are implemented from scratch in JavaScript and verified by 38 automated tests that compare them with brute-force solutions. The system is a static HTML, CSS and JavaScript application that uses Leaflet for mapping and needs no server or database.

On the real Tambaram network all four shortest-path algorithms returned the same optimal cost; averaged over 30 random routes, A* and bidirectional search settled about 38% and 66% fewer nodes than Dijkstra. In a fire scenario with the default shelter capacities (14,600 places for 47,636 people), the min-cost-flow evacuation plan never overfilled a shelter, whereas sending everyone to the nearest shelter overfilled shelters by 33,904 people; with raised capacities the plan placed 40,720 people with no overflow, against 5,888 overflowing with the nearest-shelter method.

Keywords: graph algorithms, shortest path, A*, min-cost max-flow, GIS, OpenStreetMap, flood, fire, earthquake, evacuation planning.

Table of Contents

Abstract **1**

Chapter 1 – Introduction **3**

1.1 Project Overview 3
1.2 Problem Statement 3
1.3 Objectives 3
1.4 Scope of the Project 4

Chapter 2 – System Analysis **5**

2.1 Existing System and Its Limitations 5
2.2 Proposed System and Its Advantages 5
2.3 Functional Requirements 6

Chapter 3 – System Design **7**

3.1 System Architecture 7
3.2 Workflow / Flowchart 8
3.3 Data Design (No Database) 8
3.4 Technologies and Tools Used 9

Chapter 4 – Implementation **10**

4.1 Application Modules and Their Functions 10
4.2 Implementation Details 10
4.3 Screenshots of the Application and Its Features 13

Chapter 5 – Testing and Results **22**

5.1 Test Cases and Results 22
5.2 Application Outputs 23
5.3 Results and Discussion 25

Chapter 6 – Conclusion and Future Scope **27**

6.1 Conclusion 27
6.2 Limitations 27
6.3 Future Enhancements 27

References **29**

Appendices **30**

Appendix A: Project Structure and How to Run 30
Appendix B: Assumed Model Parameters 30
Appendix C: Key Source Code 31

Chapter 1 – Introduction

1.1 Project Overview

Roads are the main way people reach hospitals, shelters and homes, and during a flood, fire or earthquake they are also the first thing to fail. A navigation tool that only knows normal conditions cannot answer the questions that matter in an emergency: which roads are still usable, how long will the trip really take, where should people go, and which road is the single point of failure.

This project builds a web-based Geographic Information System (GIS) tool for a real area, Tambaram in Chennai. It loads the real road network and nearby places from OpenStreetMap, models three hazards on top of it, and applies a library of classic graph algorithms to answer routing, service-coverage, evacuation and vulnerability questions. The user works on a map: scenarios are switched with buttons, hazards are controlled with sliders, and every result is drawn on the map with statistics.

The core design idea is that a scenario changes only the graph (which segments are blocked, which are slower, and by what time each intersection must be passed). The algorithms are therefore independent of the scenario, and the same code serves everyday routing, flood detours, fire escape and earthquake reliability.

1.2 Problem Statement

Existing consumer navigation tools optimise a single trip under normal traffic and do not model hazards, evacuation capacity or network fragility. Professional GIS software can do such analyses but requires training, licences and manual data preparation. There is a need for an easy-to-use, free, explainable tool that, for a real city area, can:

- find routes that respect road closures, one-way streets, flooding, fire spread and earthquake damage;
- compare routing algorithms on the same real data;
- plan evacuation of a population to shelters of limited capacity;
- identify the roads whose failure would isolate parts of the area.

1.3 Objectives

- Build a road graph of Tambaram from real OpenStreetMap data, with no hard-coded map values.
- Implement shortest-path algorithms from scratch (Dijkstra, A*, bidirectional Dijkstra, Bellman-Ford) with a user-selectable algorithm, and Yen's algorithm for alternative routes.
- Model flood, fire (with wind) and earthquake scenarios and apply them to the graph, including drive and walk modes and rush hour.

- Plan evacuation with min-cost max-flow, and compare it with the naive nearest-shelter assignment.
- Analyse service areas (multi-source Dijkstra) and suggest new shelter locations (greedy maximum coverage).
- Find weak points of the network with Tarjan's bridge algorithm and Brandes betweenness centrality.
- Estimate earthquake reliability with Monte Carlo simulation, and plan multi-stop trips (TSP).
- Verify every algorithm with automated tests against brute-force solutions, and measure their performance on real data.

1.4 Scope of the Project

In scope: the road network, places and buildings of a bounding box around Tambaram (default 12.900°N–12.950°N, 80.060°E–80.130°E); drive and walk travel; flood, fire and earthquake scenarios; routing, service areas, evacuation planning, network analysis and trip planning; a map-based user interface that works in a desktop browser.

Out of scope: live traffic data, real census data, building-level damage, structural engineering models, real-time sensor feeds, and multi-city coverage. The hazard models are deliberately simple and explainable; they are intended to demonstrate algorithms, not to replace engineering studies. The area can be changed by running the data-download script with a different bounding box.

Chapter 2 – System Analysis

2.1 Existing System and Its Limitations

Two kinds of systems are commonly used for the problems above.

Consumer navigation applications give fast routes under normal and traffic conditions. Their limitations for emergency planning are that the algorithm is a black box, hazards such as a user-defined flood level or a spreading fire cannot be modelled, there is no evacuation capacity model, and nothing reports the structural weak points of the network.

Desktop GIS packages offer network analysis tools but need expert training, licences or heavy installation, and data preparation is manual. They are unsuitable for quickly exploring "what if the water rises by one metre" or showing how an algorithm behaves.

Common limitations of both: no side-by-side comparison of algorithms, no combined treatment of several hazards on the same data, and little explanation of why a result was produced.

2.2 Proposed System and Its Advantages

The proposed system is a single-page web application with a scenario layer, five analysis tools and a map. Table 2.1 compares it with the existing approaches.

Table 2.1: Comparison of the proposed system with existing approaches

Aspect	Navigation app	Desktop GIS	Proposed system
Cost and setup	Free, no setup	Licence and training	Free, open the HTML file
Hazards	Traffic only	Possible with manual work	Flood, fire, earthquake, rush hour, toggled with one click
Algorithm choice	Hidden	Fixed tools	Dijkstra, A*, bidirectional, Bellman-Ford, Yen; live comparison
Evacuation capacity	No	Manual modelling	Min-cost max-flow with shelter capacities
Weak-point analysis	No	Possible	Tarjan bridges and betweenness in one click
Data	Proprietary	User supplied	Open data (OpenStreetMap, SRTM), downloaded by a script
Explainability	Low	Medium	Statistics shown: nodes settled, time, cost, coverage

Advantages: it runs entirely in the browser, needs no server or database, works offline once the data is downloaded, uses real open data with no hard-coded values, and every algorithm was written from scratch so its behaviour can be inspected and explained.

2.3 Functional Requirements

Table 2.2: Functional requirements

ID	Requirement
FR1	Load a real road network, places, buildings and elevation from a generated data file; no map values are hard-coded in the code.
FR2	Let the user pick an origin and destination on the map and display the route, distance, travel time and step-by-step directions.
FR3	Let the user choose the algorithm (A*, Dijkstra, bidirectional Dijkstra, Bellman-Ford), the metric (time or distance), and the number of alternative routes (1 to 5).
FR4	Support drive mode (one-way streets, class-based speeds) and walk mode, and a rush-hour option.
FR5	Flood scenario: water-level slider, optional flood source with connected flooding, bridges stay open, shallow wading on foot.
FR6	Fire scenario: ignition points, time since ignition, spread speed, wind direction and strength, safety margin; routes must reach each intersection before the fire.
FR7	Earthquake scenario: epicentre, magnitude, road fragility, random damage draw, Monte Carlo reliability.
FR8	Find the nearest hospital, shelter or fire station from a point under the current scenario.
FR9	Compare all algorithms on the current pair and benchmark them on random pairs.
FR10	Service areas: show the catchment of a facility type within a time limit and the share of population covered.
FR11	Evacuation: assign zones to shelters under capacity limits, compare with nearest-shelter assignment, show road load, and suggest new shelter sites.
FR12	Network analysis: bridges, articulation points and critical roads (betweenness).
FR13	Multi-stop round trip using nearest neighbour, 2-opt and exact Held-Karp.
FR14	Show clear messages when no route exists, with the reason.

Non-functional requirements: responsiveness (a route is computed in a few milliseconds on the real network), correctness (verified by automated tests), portability (any modern desktop browser, opens from a local file), and maintainability (separate modules for graph, algorithms, scenarios and interface).

Chapter 3 – System Design

3.1 System Architecture

The system has two parts (Figure 3.1). An offline download script (Node.js) fetches the data once and writes a single data file. The browser application then loads that file, builds the graph in memory and runs everything locally.

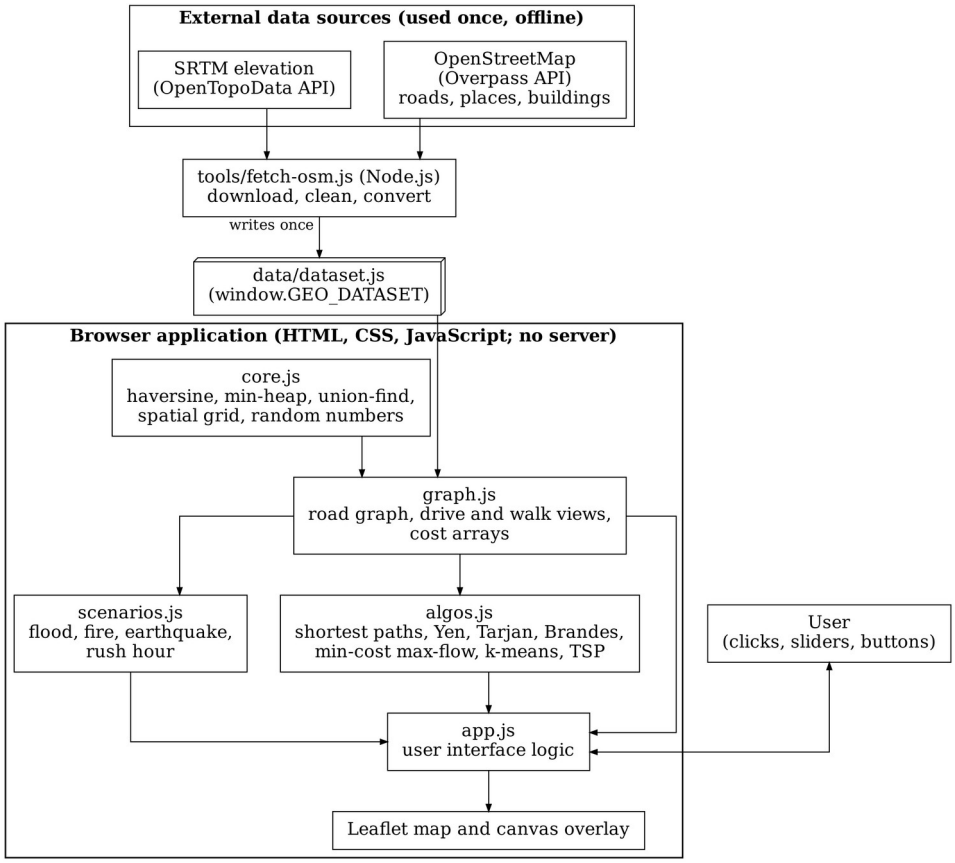


Figure 3.1: System architecture / block diagram

The responsibilities of the blocks are:

- **fetch-osm.js** downloads roads, places and buildings from the Overpass API and elevation from OpenTopoData, cleans them and writes `data/dataset.js`.
- **core.js** provides shared building blocks: haversine distance, a binary min-heap, union-find, a spatial grid for nearest-node lookup and a seeded random number generator.
- **graph.js** converts the dataset into the road graph and builds compact adjacency views (CSR format) for driving and walking, plus cost arrays for a given scenario and metric.
- **scenarios.js** implements the flood, fire, earthquake and rush-hour models.
- **algos.js** contains all algorithms.
- **app.js** connects the buttons and sliders to the algorithms and draws results on a Leaflet map with a custom canvas layer.

3.2 Workflow / Flowchart

Figure 3.2 shows the workflow. After the graph is built, the user chooses a scenario and travel mode. The scenario produces blocked segments, delay multipliers and (for fire) deadlines. A cost array is built from them, the user runs one of the five tools, and the result is drawn. Changing a slider or a point re-runs the pipeline, so routes re-plan live.

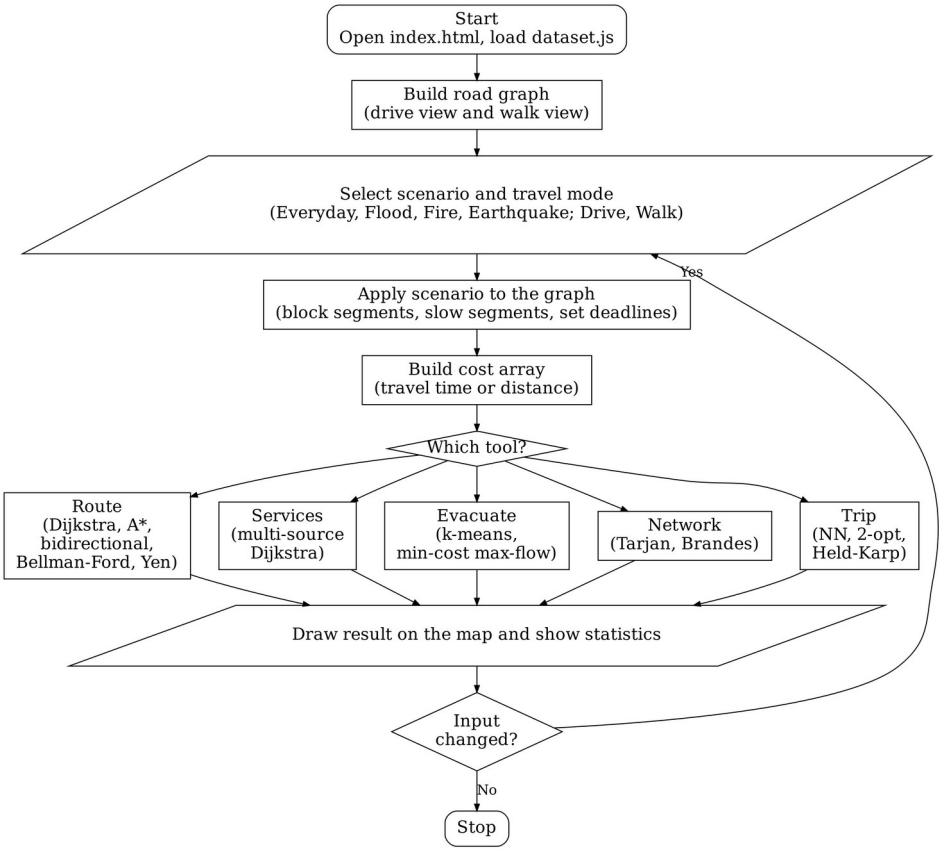


Figure 3.2: Application workflow

For the fire scenario the pipeline has one extra step. Fire arrival time at every intersection is computed once (multi-source Dijkstra). A route may enter an intersection only if the current time plus the time already travelled is smaller than that intersection's arrival time minus the safety margin. This is a time-dependent shortest-path search, and it is why bidirectional search (which starts from the destination and cannot know the arrival time) is replaced by A* in the fire scenario.

3.3 Data Design (No Database)

The project needs no database. The map data never changes while the application runs, so a single static file is simpler and faster than a database server. `tools/fetch-osm.js` writes `data/dataset.js`, which assigns one object to `window.GEO_DATASET`. Because it is a script file, the page also works when opened directly from disk. Table 3.1 shows its structure.

Table 3.1: Structure of the dataset file (equivalent of the schema)

Field	Content	Used for
meta	name, bounding box, source, fetch date, flags	title, map extent, attribution
nodes	array of [latitude, longitude] for every road point	graph nodes
ways	list of {n: node indices, h: road class, o: one-way code, b: bridge flag, nm: name}	graph edges, speeds, directions
pois	list of {lat, lon, type, name} (hospital, clinic, school, college, fire station, community centre, shelter, assembly point, police)	facilities and shelters
buildings	array of [lat, lon] building centroids	population estimate
elev	{south, west, north, east, rows, cols, v[]} coarse elevation grid	flood model (interpolated)

Inside the browser the dataset is converted into typed arrays: node coordinates, segment endpoints, lengths, classes, one-way flags, bridge flags and elevation, plus a compressed sparse row (CSR) adjacency structure for each travel mode. This layout is cache-friendly and lets Dijkstra-type searches run in a few milliseconds on a 14,000-node graph. The entity-relationship view is simple: a way is made of many segments, a segment joins two nodes, and a place is attached to its nearest node.

3.4 Technologies and Tools Used

Table 3.2: Technologies and tools

Technology	Purpose
HTML5, CSS3	Page structure and styling (own colour palette taken from road-safety signage)
JavaScript (ES2015+)	All application and algorithm code; the algorithm modules also run under Node.js for testing
Leaflet 1.9.4	Interactive map, markers, tooltips, zoom and pan (library kept in the project, works offline)
HTML5 Canvas	Custom overlay that draws about 15,000 road segments, hazards and routes quickly
Node.js 18+	Data download script and automated tests
OpenStreetMap (Overpass API)	Real roads, places and buildings (open data)
SRTM 30 m via OpenTopoData	Elevation for the flood model
Esri / OpenStreetMap tiles (optional)	Background map; the application also works without tiles
Graphviz, Python (Pillow)	Used only to prepare diagrams for this report

Chapter 4 – Implementation

4.1 Application Modules and Their Functions

Table 4.1: Modules of the application

Module (file)	Main functions	Lines
core.js	haversine, bearing, MinHeap, UnionFind, SpatialGrid, seeded RNG	174
graph.js	buildGraph, buildView (drive and walk, CSR), makeCosts, arcStats, elevation interpolation	185
algos.js	sssp (multi-source / reverse), dijkstra, astar, bidirectional, bellmanFord, yen, tarjan, edgeBetweenness, minCostMaxFlow, kmeans, greedyCoverage, TSP (NN, 2-opt, Held-Karp)	579
scenarios.js	flood, fireArrival, fireBlocked, quakeProb, sampleFailures, quakeMonteCarlo, rushDelay	158
app.js	map and canvas overlay, scenario panel, routing, compare and benchmark, services, evacuation, placement, network analysis, Monte Carlo, trip	935
tools/fetch-osm.js	Overpass and elevation download, cleaning, writing the dataset	243
tools/test.js, test-fetch.js	38 algorithm tests and the data-parsing tests	433

The interface has a scenario panel (Everyday, Flood, Fire, Earthquake; Drive or Walk; rush hour) and five tool tabs. Table 4.2 summarises the tabs.

Table 4.2: User interface tabs

Tab	What the user does	Algorithms used
Route	Set A and B, choose algorithm, metric and number of routes; compare algorithms; benchmark	Dijkstra, A*, bidirectional, Bellman-Ford, Yen
Services	Choose facility type and time limit; see catchment and % population covered	Multi-source reverse Dijkstra
Evacuate	Set people per building, zones and shelter capacities; compute plan; suggest new shelters	k-means++, min-cost max-flow, greedy max-coverage
Network	Find bridges and critical roads	Tarjan, Brandes betweenness
Trip	Add stops and plan a round trip	Nearest neighbour, 2-opt, Held-Karp

4.2 Implementation Details

4.2.1 Building the road graph

Every OpenStreetMap way with a highway tag is split into segments between consecutive nodes. Only the largest connected component is kept, so every node can reach every other node when no scenario is active. Each segment stores its length (haversine), road class, one-way flag and bridge flag. A directed arc list is produced for each travel mode: in drive mode a one-way street gives one arc, in walk mode every street gives two arcs. The base cost of an arc is its length divided by a speed that depends on the road class (assumed free-flow speeds from 70

km/h on motorways to 10 km/h on living streets; walking 1.35 m/s). A reverse adjacency is stored as well, which bidirectional search and multi-source searches need.

4.2.2 Shortest-path algorithms

- **Dijkstra** uses a binary min-heap with lazy deletion. The same routine supports several sources, reverse search, a cost cut-off, blocked nodes and per-node deadlines, so it is reused by several features.
- **A*** orders the search by g(n) + h(n), where h is the haversine distance to the destination multiplied by the cheapest cost per metre in the network. This heuristic never over-estimates and is consistent, so A* returns the optimal route and settles each node once.
- **Bidirectional Dijkstra** searches forward from the origin and backward from the destination (on the reverse graph), always expanding the smaller frontier, and stops when the sum of the two smallest keys is at least the best meeting cost found.
- **Bellman-Ford** relaxes all arcs in rounds with early exit. It is O(VE) and is included as the baseline that shows why the heap-based algorithms are needed.
- **Yen's algorithm** produces the k best loop-free routes: for each node of the previous best route it blocks the arcs already used from that node and the nodes of the root path, runs an A* spur search, and keeps the best candidates.

4.2.3 Scenario models

Flood. An intersection is flooded if its elevation (interpolated from the SRTM grid) is at or below the water level. With "connected flooding" enabled, water spreads by breadth-first search from the chosen source through flooded intersections only, so isolated low spots stay dry. A segment is cut if either end is flooded, except bridges. On foot, a flooded segment whose average depth is at most 0.5 m is passable at one third of the speed; deeper segments are blocked.

Fire. Fire arrival time is the shortest-path distance from the ignition points over the walking graph, where the spread rate along an arc depends on wind: rate = $v \times (1 + k \times \cos(\text{bearing} - \text{wind direction}))$, with a floor of 10% of v. Segments reached by the current time are blocked. For routing, each intersection gets the deadline (arrival time − safety margin); an intersection may be entered only if the current time plus the travel time so far is smaller than its deadline.

Earthquake. Shaking intensity at a segment is $I = M - 3.2 \times \log10(d + 0.5) - 4.6$, where M is the magnitude and d the distance from the epicentre in kilometres. The failure probability is $p = \min(0.95, \text{logistic}(1.5 \times I) \times \text{fragility} \times m)$, where m is 15 for bridges and 1 otherwise. One random draw gives the red segments on the map. Monte Carlo repeats the draw many times (default 100). For each draw it uses union-find on the surviving segments to test (a) whether the origin and destination are still connected, (b) whether the planned route is intact, and (c) which zones are cut off from every shelter and hospital.

Rush hour. Each road class has a slowdown multiplier (from 1.0 on living streets to 1.9 on trunk and primary roads) applied to travel time in drive mode.

4.2.4 Service areas

A single reverse Dijkstra search is started from all facilities of the chosen type at once, with a time cut-off. Every intersection then knows the travel time to its nearest facility and which facility that is. Road segments are coloured by thirds of the time limit, and the percentage of the estimated population within the limit is computed from the population zones.

4.2.5 Evacuation planning

Population is estimated by grouping the building locations into zones with k-means++ (25 iterations, fixed seed) and multiplying the building count of each zone by the "people per building" setting. This is an assumption, because OpenStreetMap has no census data. A flow network is built with a source, one node per zone, one node per shelter and a sink. The source-to-zone arcs have the zone population as capacity, the zone-to-shelter arcs have the travel time as cost, and the shelter-to-sink arcs have the shelter capacity. The min-cost max-flow is found by successive shortest paths using SPFA. The result is compared with the naive assignment in which every zone goes to its nearest shelter regardless of capacity. Road load is obtained by adding the flow of each assignment along its shortest path.

For new shelter sites, a candidate site covers a zone if the zone can reach it within the chosen time (a bounded reverse Dijkstra). Candidates are all zone centres plus up to 300 random intersections. The greedy maximum-coverage algorithm repeatedly picks the site that covers the most still-uncovered people, which guarantees at least $1 - 1/e$ (about 63%) of the best possible coverage. It is compared with the average of 200 random choices.

4.2.6 Network vulnerability

Tarjan's low-link depth-first search (written iteratively to avoid stack overflow) finds all bridge segments and articulation points of the undirected network in $O(V + E)$, and reports how many intersections each bridge would cut off. Brandes' algorithm computes edge betweenness: from each sampled source (default 80) a weighted shortest-path search counts the shortest paths through each edge, and the dependencies are accumulated backwards. Edges with the highest scores carry the most shortest-path traffic.

4.2.7 Multi-stop trips

Travel times between all pairs of stops are obtained from one Dijkstra search per stop. Three tour methods are then applied: nearest neighbour, 2-opt improvement of that tour, and the exact Held-Karp dynamic programme, which stores the cheapest way to visit each subset of stops ending at each stop ($O(n^2 2^n)$, limited to 12 stops).

4.2.8 Map rendering

Drawing 15,000 road segments as Leaflet vector layers is slow, so a custom canvas overlay was written. Coordinates are projected once to normalised Web Mercator, only segments inside the viewport are drawn, and all segments of one style are drawn in a single path. Hazard, route, load and explored-node layers are separate named layers, so they can be switched independently.

4.2.9 Complexity summary

Table 4.3: Time complexity of the algorithms (V intersections, E segments)

Algorithm	Time	Remark
Dijkstra (binary heap)	O((V + E) log V)	optimal, non-negative weights
A*	same worst case	far fewer nodes with a good heuristic
Bidirectional Dijkstra	O((V + E) log V)	about half the nodes in practice
Bellman-Ford	O(V E)	baseline
Yen (k paths)	O(k V (V + E) log V)	A* spur searches
Multi-source Dijkstra	O((V + E) log V)	one pass for all facilities
Tarjan bridges	O(V + E)	iterative DFS
Brandes betweenness	O(k (V + E) log V)	k sampled sources
Min-cost max-flow (SSP)	O(F × SPFA)	F = augmenting paths
k-means++	O(i n k)	i iterations
Greedy max-coverage	O(k C Z)	at least (1 − 1/e) of optimum
Held-Karp	$O(n^2 2^n)$	exact TSP, n up to 12

4.3 Screenshots of the Application and Its Features

Figure 4.1 shows the data download step. The remaining screenshots were taken from the working application on the real Tambaram data (14,022 intersections, 15,402 segments, 11,909 buildings, 134 places, elevation 9–57 m at the road intersections).

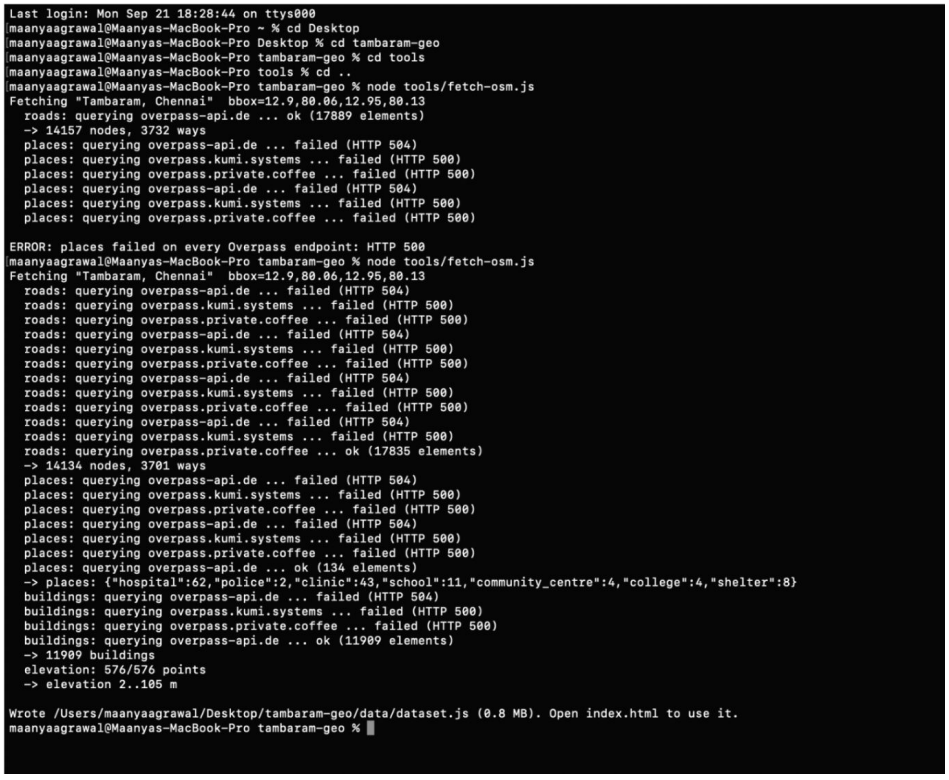


Figure 4.1: Downloading the real data with node tools/fetch-osm.js. The script queries the Overpass servers (some answer with HTTP 504 or 500 and the script retries on the next server), receives 14,134 nodes and 3,701 roads, 134 places, 11,909 buildings and 576 elevation points, and writes data/dataset.js (0.8 MB). The elevation

range printed here (2 to 105 m) is that of the whole download area; the application header (9 to 57 m) reports the range at the road intersections.

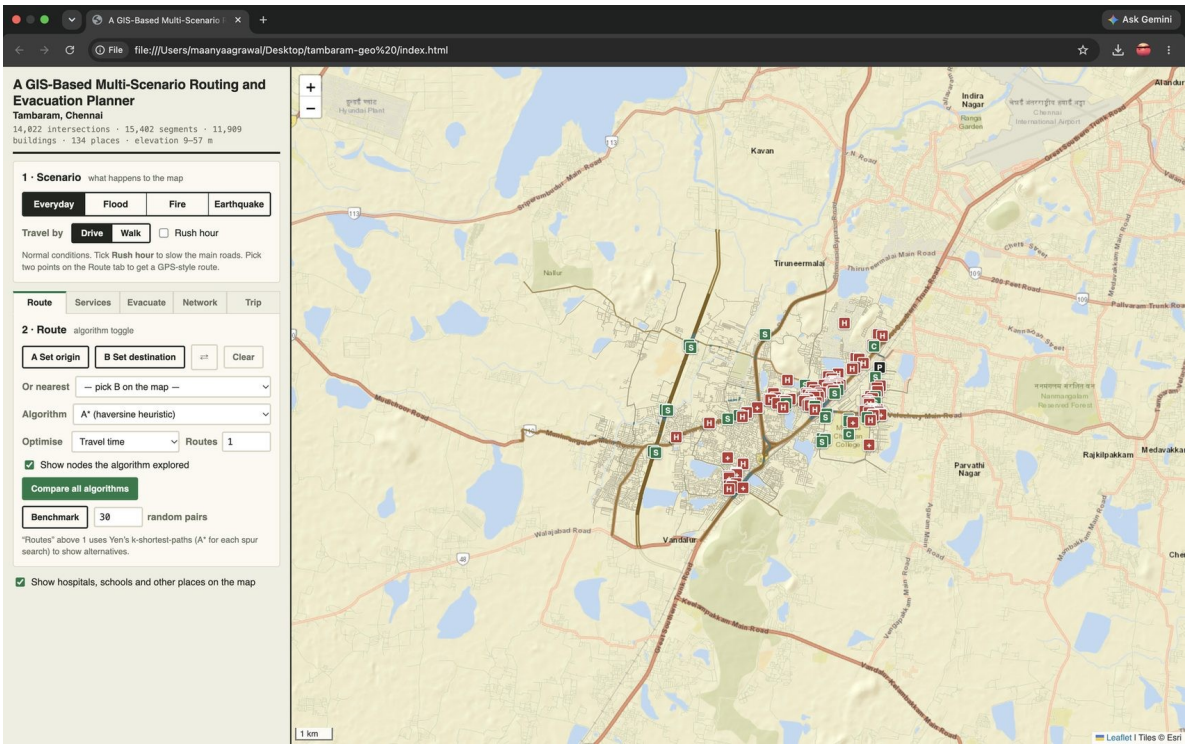


Figure 4.2: Main screen after the Tambaram data has loaded. The left panel holds the scenario selector (Everyday, Flood, Fire, Earthquake), the travel mode and the five tool tabs (Route, Services, Evacuate, Network, Trip); on the map, the red and green markers show hospitals, clinics, schools and other public places.

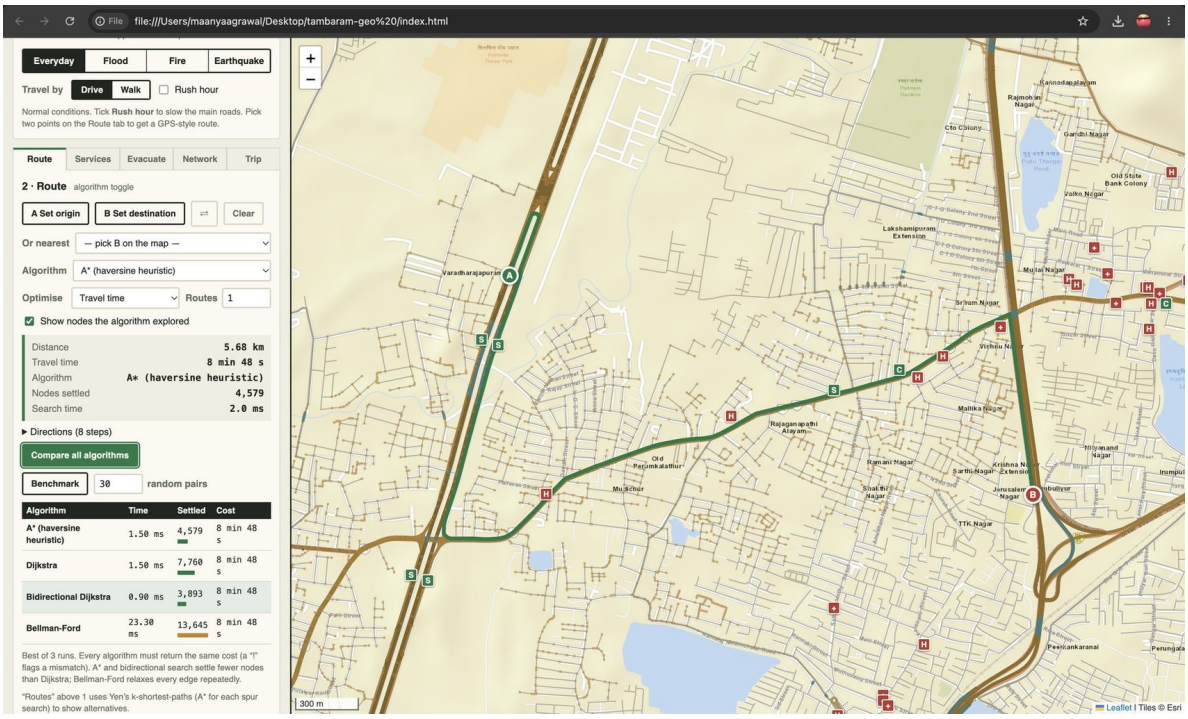


Figure 4.3: Everyday routing with A. The green line is the route from A to B (5.68 km, 8 min 48 s) and the small dots are the nodes the search explored. The table compares the four algorithms on this pair: all return the same cost, but they settle 4,579 (A*), 7,760 (Dijkstra), 3,893 (bidirectional) and 13,645 (Bellman-Ford) nodes.*

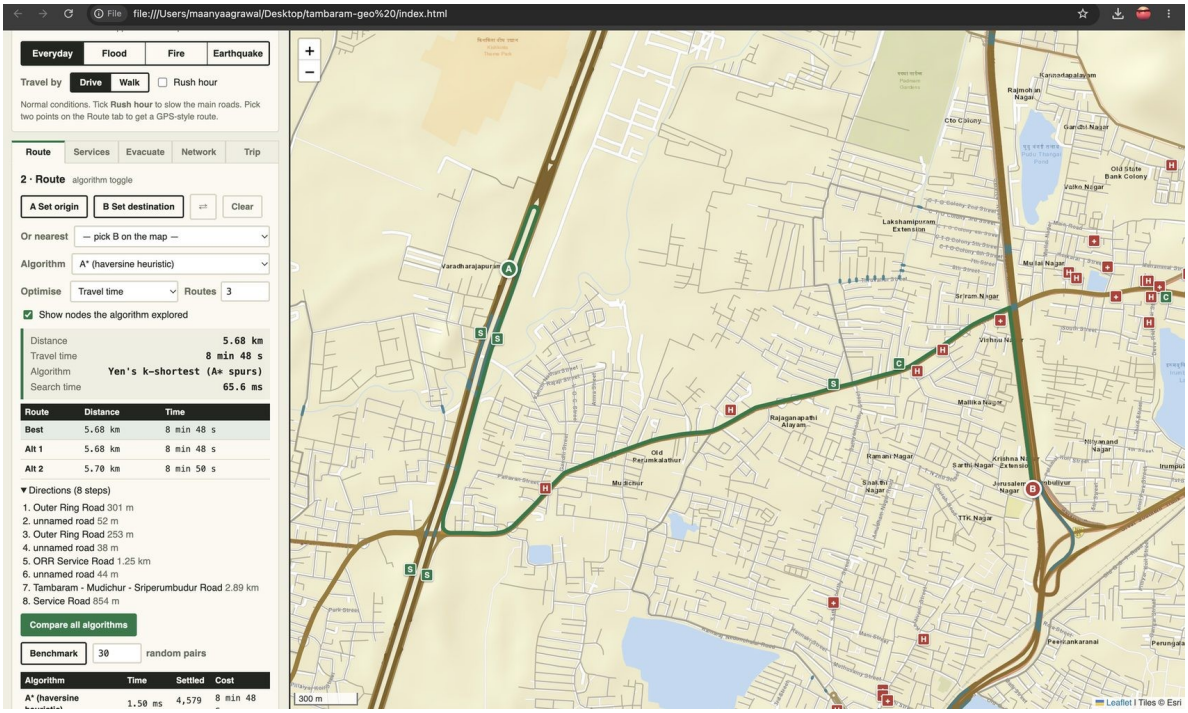


Figure 4.4: Alternative routes with Yen's algorithm (Routes = 3) for the same pair. The table lists the best route (5.68 km, 8 min 48 s) and two alternatives (5.68 km, 8 min 48 s and 5.70 km, 8 min 50 s); the step-by-step directions of the best route are shown below it.

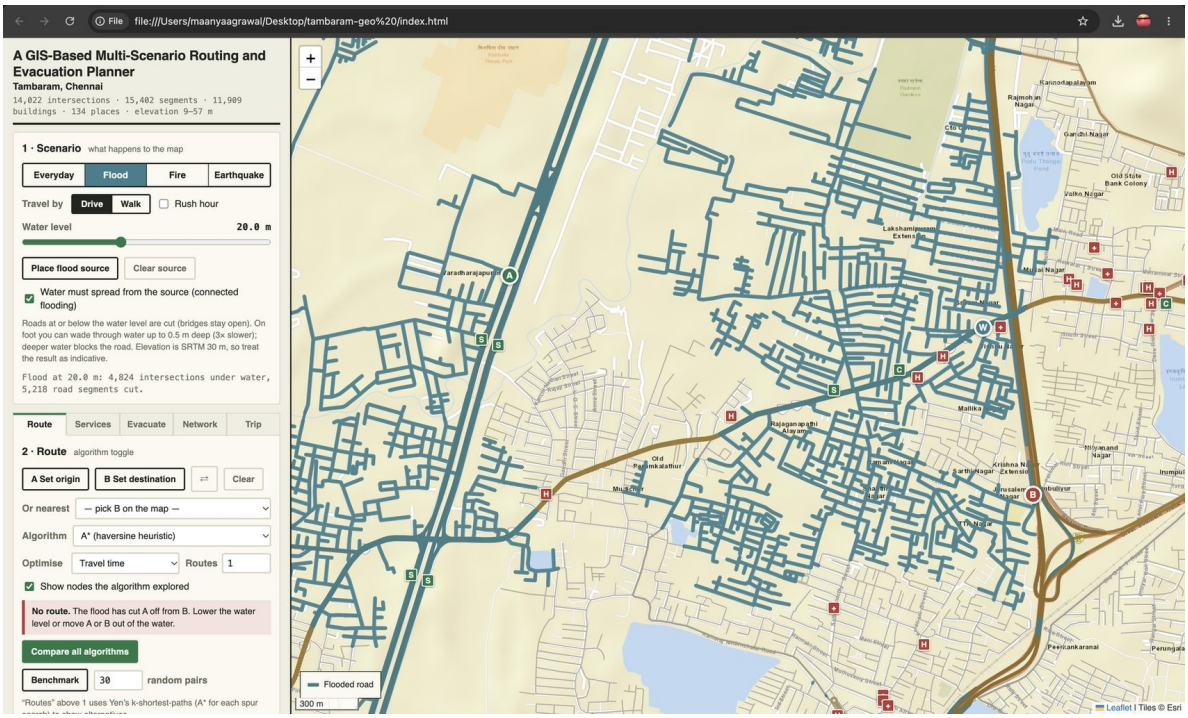


Figure 4.5: Flood scenario at a water level of 20.0 m with a flood source (W). Teal roads are cut: 4,824 intersections are under water and 5,218 segments are cut. The route reports "No route" because the flood has cut A off from B.

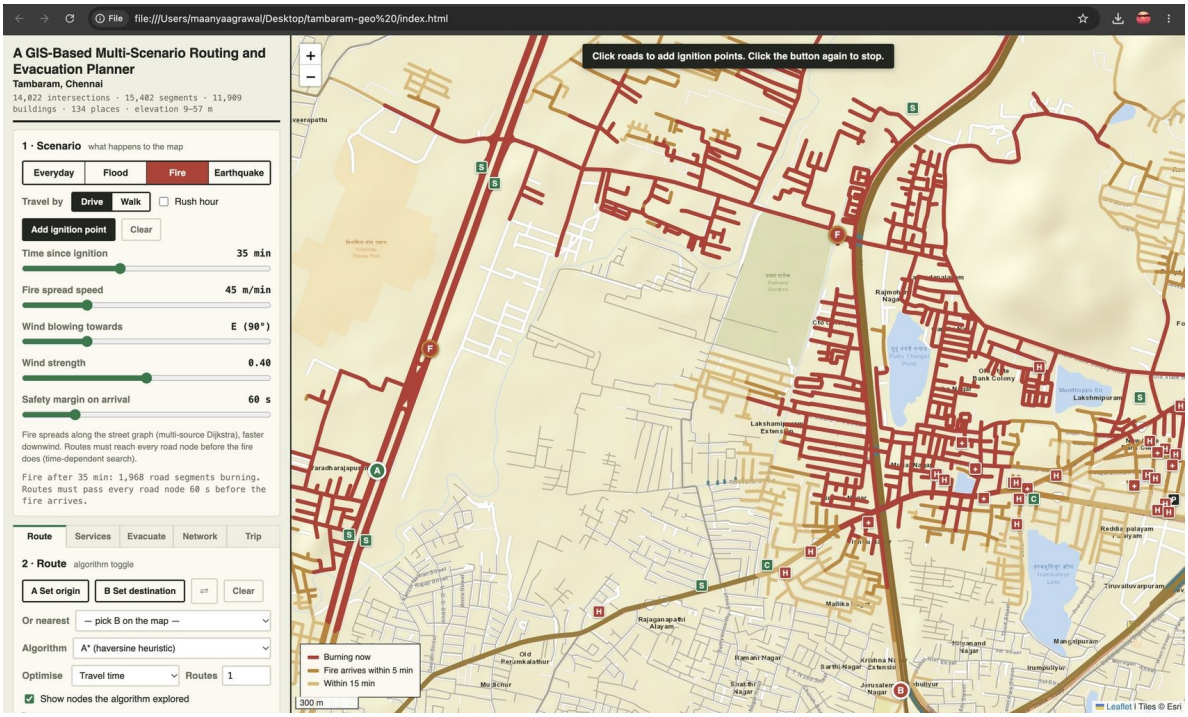


Figure 4.6: Fire scenario 35 minutes after ignition (spread 45 m/min, wind towards the east with strength 0.40, safety margin 60 s). Ignition points are marked F and 1,968 road segments are burning. Red = burning, dark amber = fire arrives within 5 minutes, light amber = within 15 minutes.

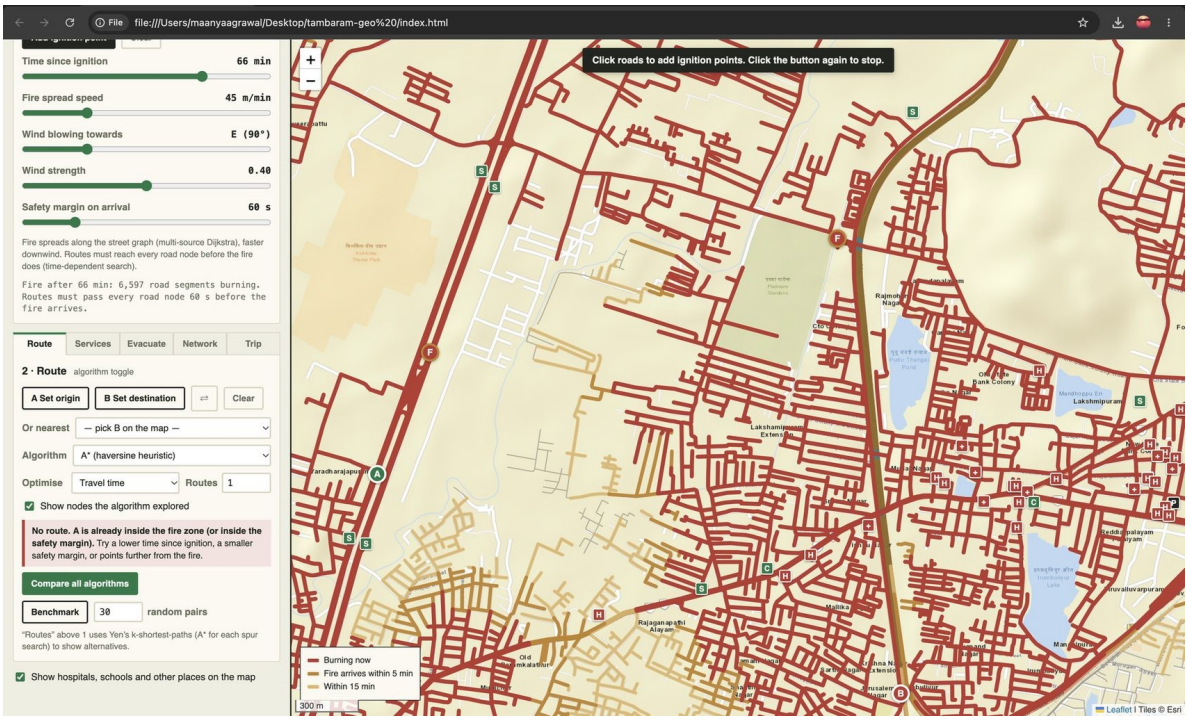


Figure 4.7: The same fire at 66 minutes. 6,597 road segments are burning, and the application reports that no route exists because A is already inside the fire zone (or inside the safety margin).

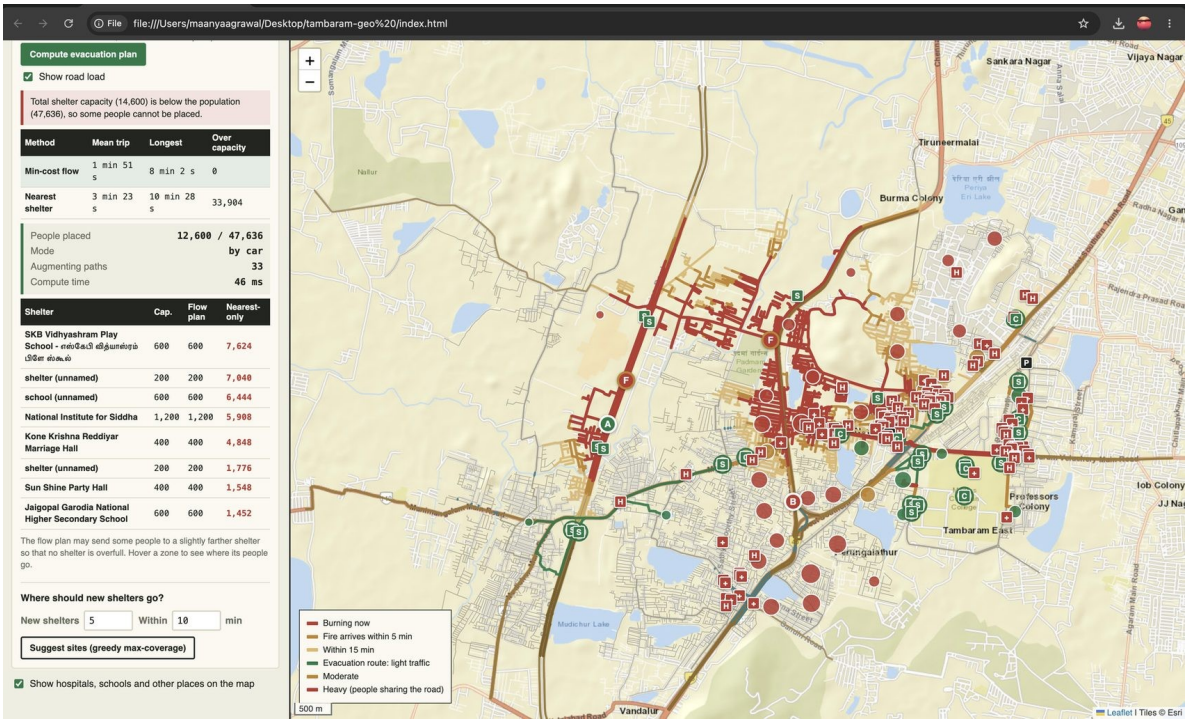


Figure 4.8: Evacuation plan under a fire 35 minutes after ignition with the default shelter capacities (school 600, college 1,200, community hall 400, other 200). Total capacity is 14,600 for 47,636 people, so only 12,600 can be placed. The min-cost flow plan overfills no shelter, while sending everyone to the nearest shelter leaves 33,904 people over capacity.

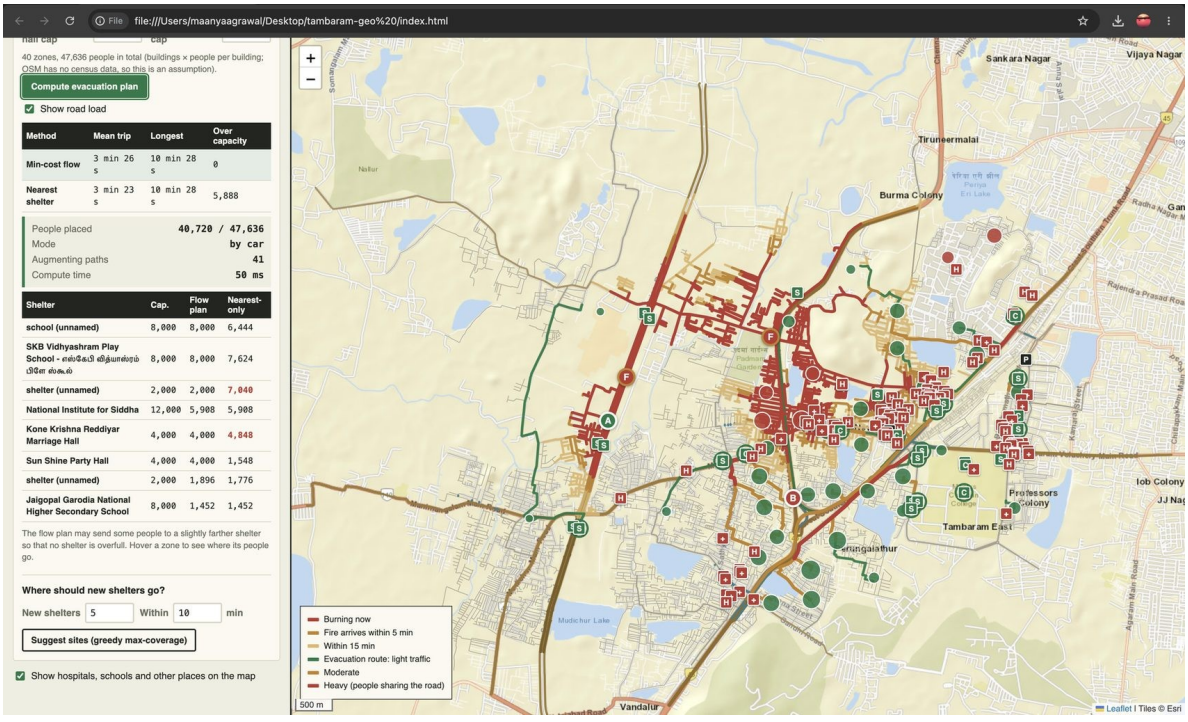


Figure 4.9: The same fire with raised shelter capacities (school 8,000, college 12,000, community hall 4,000, other 2,000). The plan places 40,720 of 47,636 people with no shelter overfilled (mean trip 3 min 26 s), whereas the nearest-shelter method overfills shelters by 5,888 people. Green circles are placed zones and the coloured roads show the evacuation load.

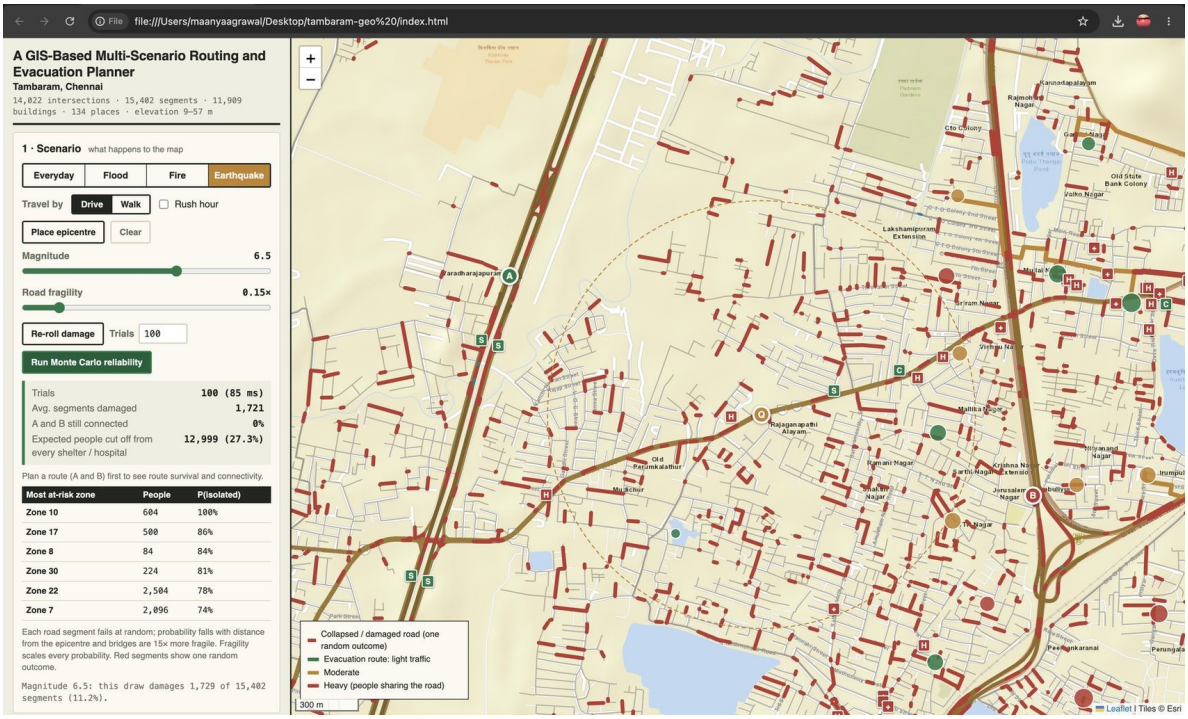


Figure 4.10: Earthquake scenario (magnitude 6.5, road fragility 0.15). The epicentre is marked Q with a dashed distance ring, and red segments are the roads damaged in one random draw (1,729 of 15,402 segments, 11.2%). The Monte Carlo run of 100 trials (85 ms) gives on average 1,721 damaged segments; A and B stay connected in 0% of the trials and 12,999 people (27.3%) are expected to be cut off from every shelter or hospital. Zone 10 is isolated in all trials.

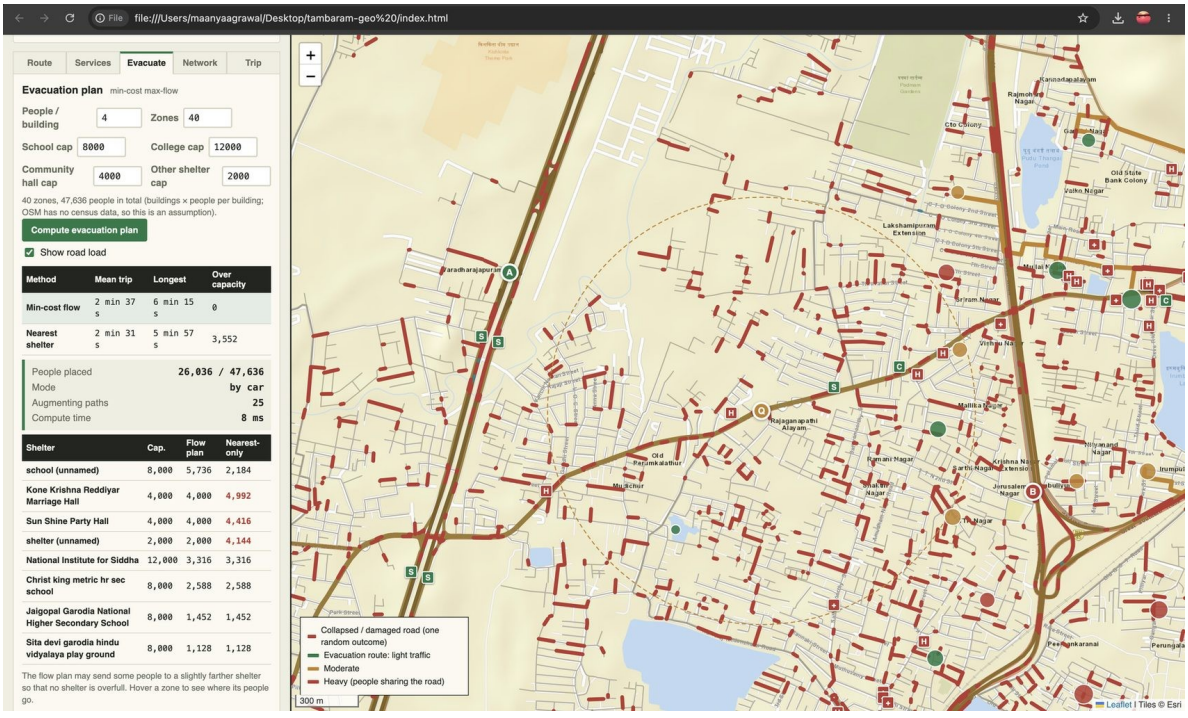


Figure 4.11: Evacuation plan after the earthquake of Figure 4.10, with raised shelter capacities. The plan places 26,036 of 47,636 people with no shelter overfilled (mean trip 2 min 37 s), whereas the nearest-shelter method overfills shelters by 3,552 people.

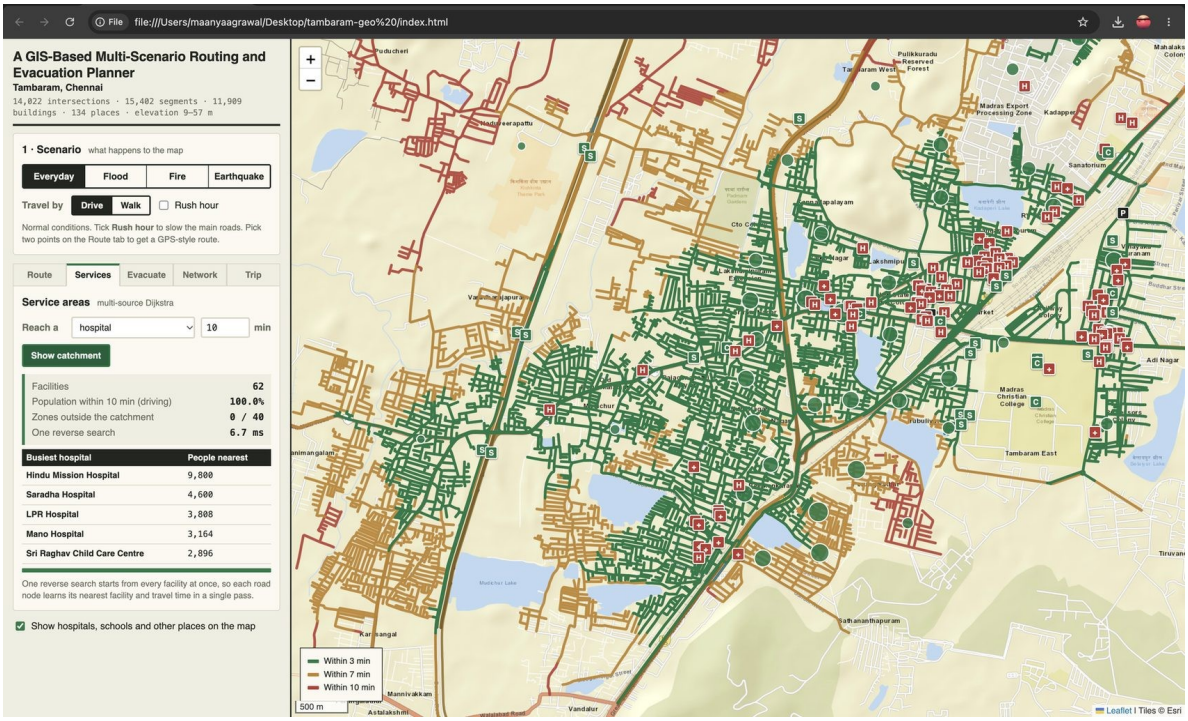


Figure 4.12: Service areas (Services tab). Catchment of hospitals within 10 minutes of driving: green within 3 minutes, amber within 7 and red within 10. All 62 hospitals are used as sources, 100% of the population and all 40 zones are covered, and one reverse search took 6.7 ms. The table lists the busiest hospitals by the number of people nearest to them.

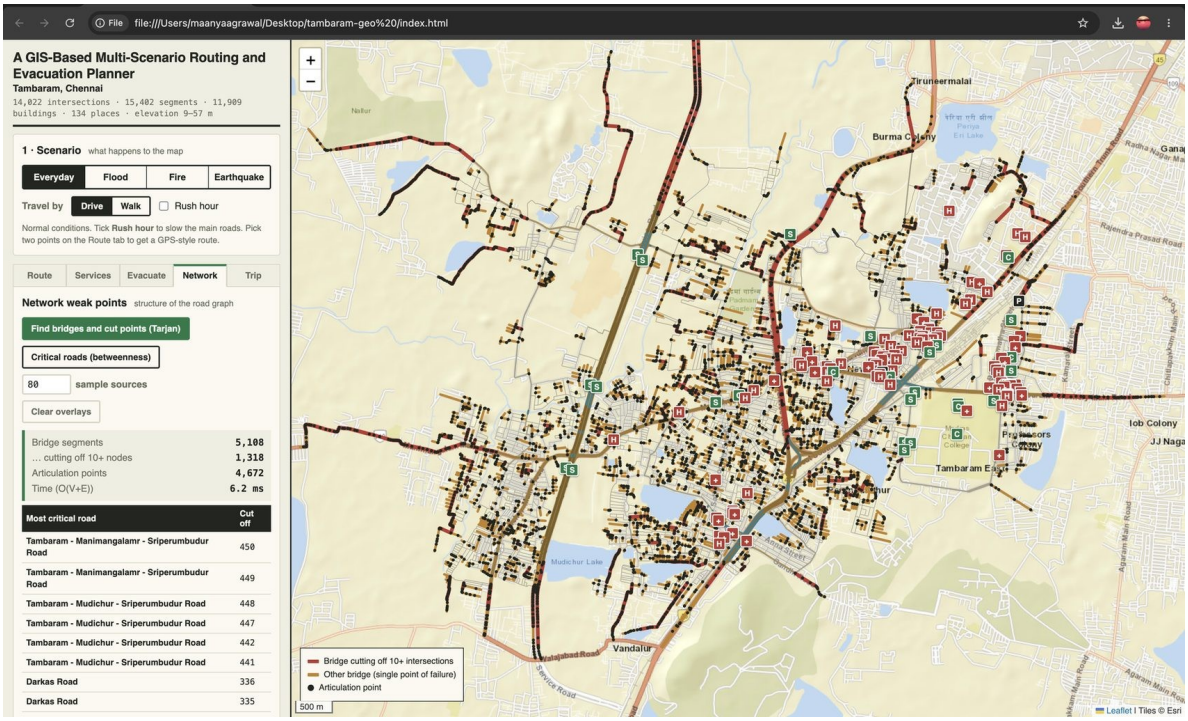


Figure 4.13: Network weak points (Network tab). Red = bridge segments that cut off 10 or more intersections, amber = other bridges (single points of failure), black dots = articulation points. Tarjan's algorithm found 5,108 bridge segments (1,318 cutting off 10 or more nodes) and 4,672 articulation points in 6.2 ms.

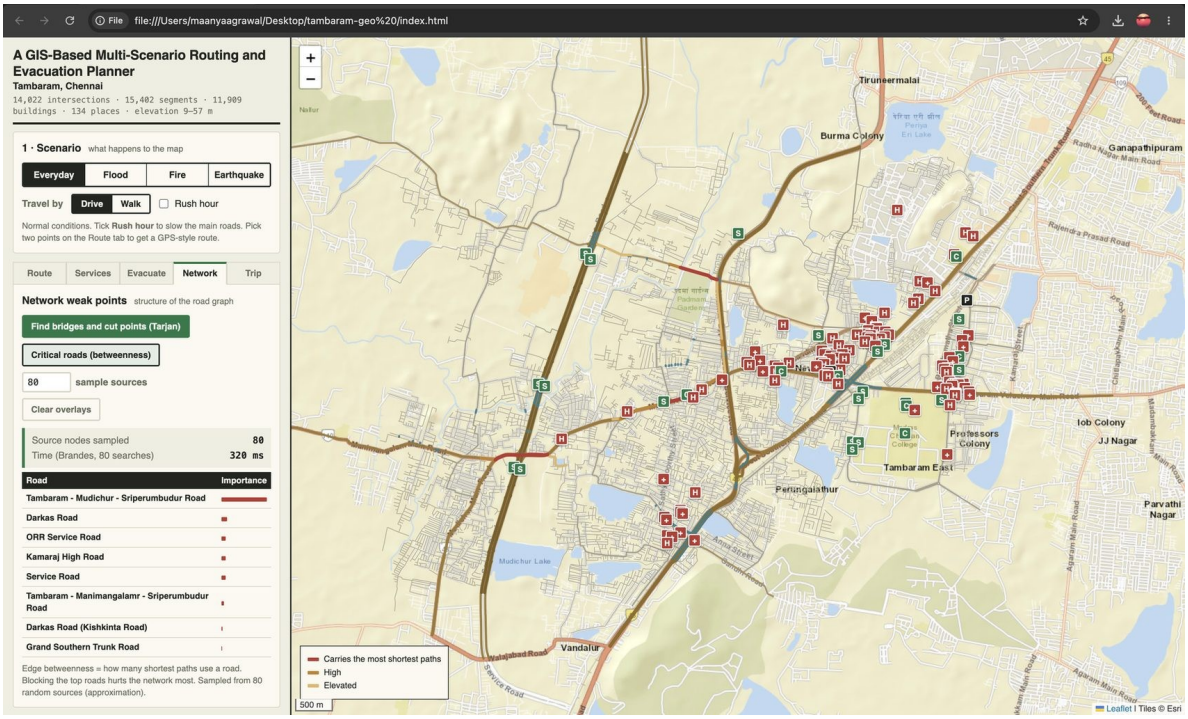


Figure 4.14: Critical roads by edge betweenness (Network tab, 80 sampled sources, 320 ms). Red marks the road that carries the most shortest paths, Tambaram-Mudichur-Sriperumbudur Road, followed by Darkas Road and the ORR Service Road.

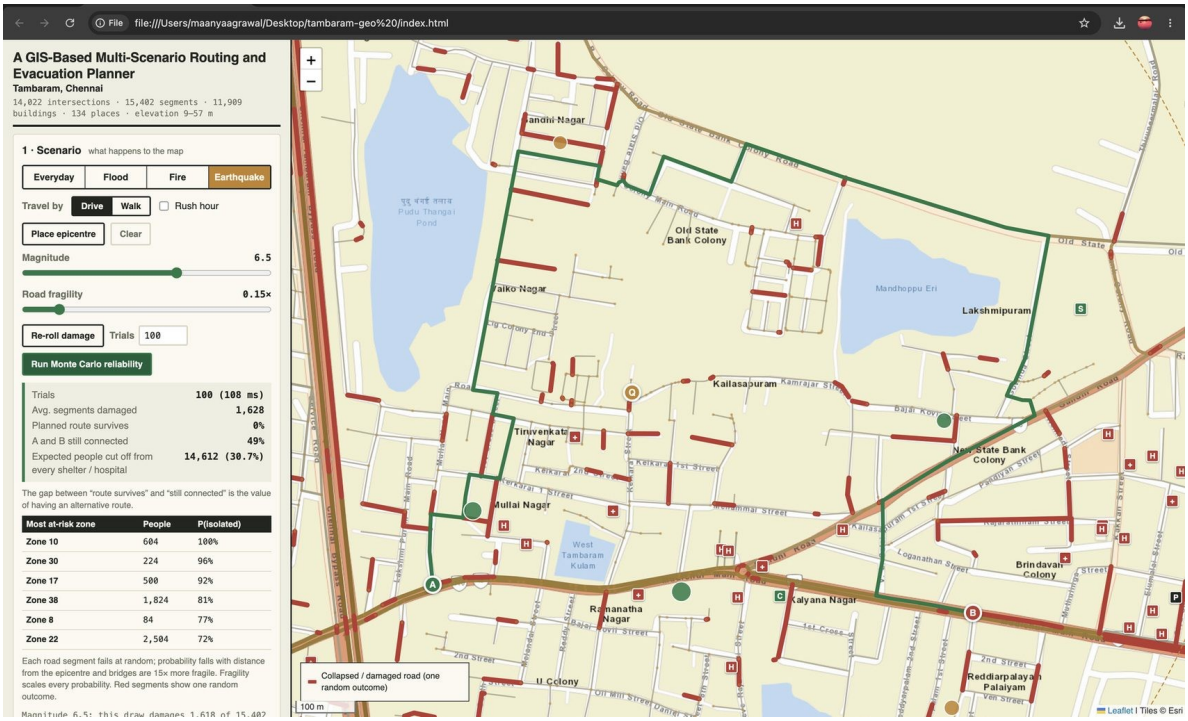


Figure 4.15: Monte Carlo earthquake reliability (magnitude 6.5, fragility 0.15, 100 trials in 108 ms). On average 1,628 segments are damaged; the planned route survives in 0% of the trials, A and B stay connected in 49%, and 14,612 people (30.7%) are expected to be cut off from every shelter or hospital. Zone 10 is isolated in all trials.

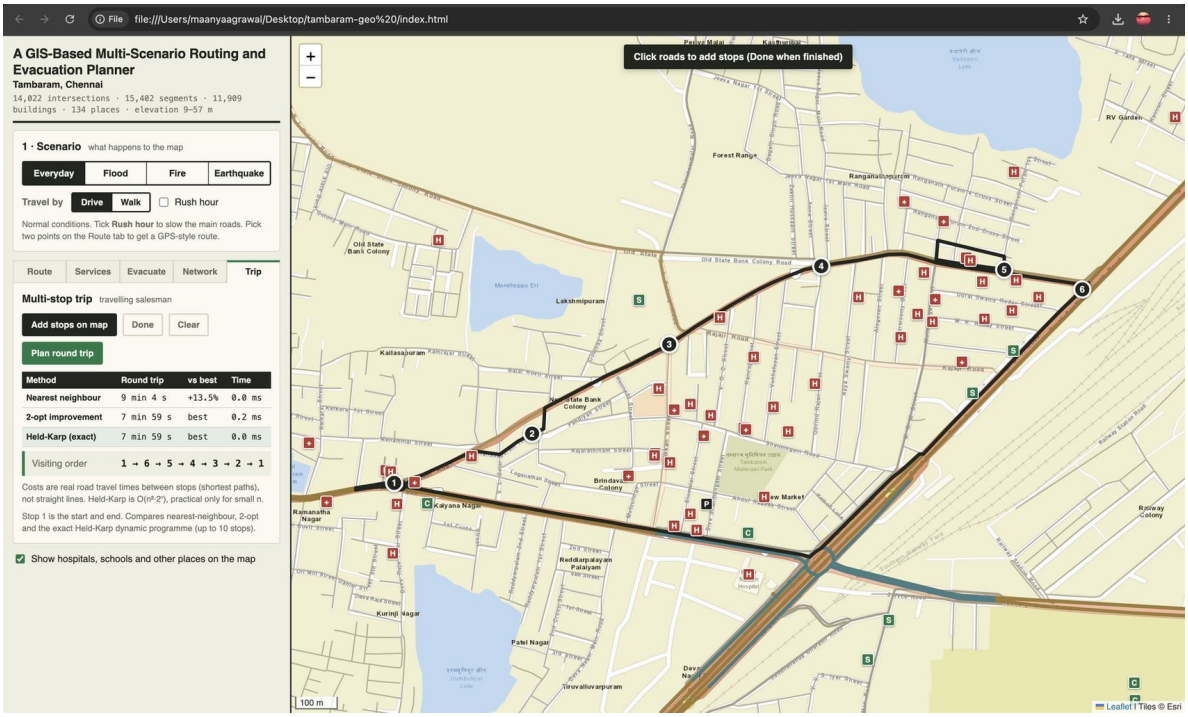


Figure 4.16: Multi-stop trip (Trip tab) with six stops. Nearest neighbour gives a 9 min 4 s round trip (13.5% longer), while 2-opt and the exact Held-Karp algorithm both give 7 min 59 s with the visiting order 1, 6, 5, 4, 3, 2, 1.

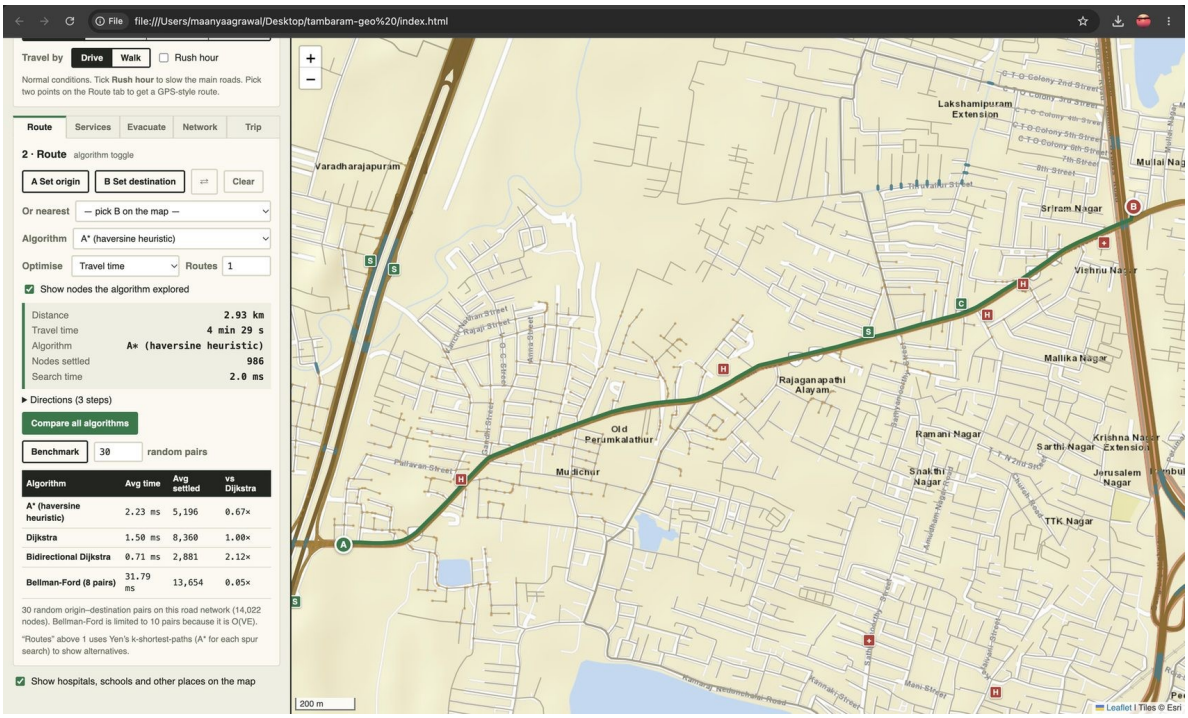


Figure 4.17: Benchmark of the four shortest-path algorithms on 30 random origin-destination pairs of the real road network (Route tab, Benchmark). The route drawn is a single A query of 2.93 km (4 min 29 s) that settled 986 nodes in 2.0 ms.*

Chapter 5 – Testing and Results

5.1 Test Cases and Results

Testing has two levels. First, 38 automated tests (`node tools/test.js`) check every algorithm against an independent method (brute force or a different algorithm) on a deterministic test graph of 28,232 intersections and 52,754 segments. Second, manual tests on the real Tambaram data check the user-visible features.

Table 5.1: Automated test cases (all 38 passed)

Group	What is checked	Result
Graph construction (5)	largest component fully connected; bridge segments present; elevation sampled; spatial grid nearest node equals brute force on 200 queries	Pass
Shortest paths (10)	Dijkstra, A*, bidirectional and Bellman-Ford return identical optimal costs and valid paths for drive/time (120 routes), drive/distance (60), walk/time (60), 12% blocked roads with delays (99), and fire deadlines; 0 mismatches	Pass
Search effort (2)	A* and bidirectional settle fewer nodes than Dijkstra (average of 120 random routes)	Pass
Yen (2)	top-5 costs equal brute-force enumeration of simple paths; city graph paths sorted, distinct, loop-free	Pass
Tarjan (4)	bridges equal brute-force edge removal (447 edges); isolated-node counts exact; articulation points equal brute-force node removal; river bridges found	Pass
Brandes (1)	edge betweenness identical to Floyd-Warshall brute force on 6 random graphs	Pass
Min-cost max-flow (1)	flow and cost equal the exhaustive optimum on 150 small instances	Pass
TSP (2)	Held-Karp optimal against all permutations; 2-opt never worse than nearest neighbour and never better than optimal	Pass
k-means, coverage (2)	k-means recovers 3 separated clusters; greedy coverage at least (1 − 1/e) of optimum on 60 random cases	Pass
Scenarios (rest)	flood only grows as the level rises; connected flooding is a subset of unrestricted flooding; bridges not flooded; fire spreads faster downwind than upwind (east 15.2 min, west 41.4 min); time-dependent routes always arrive before the fire (21 routes); zero fragility means everything survives; bridges more fragile than roads; Monte Carlo reproducible	Pass

Table 5.2: Manual test cases on the real Tambaram data

No.	Test	Expected	Observed
1	Everyday route A to B with A*	Valid route with statistics	5.68 km, 8 min 48 s, 4,579 nodes settled, 8 direction steps
2	Compare all algorithms on the same pair	Identical cost for all four	All four: 8 min 48 s
3	Routes = 3 (Yen)	Three distinct loop-free routes, costs non-decreasing	5.68 km / 8 min 48 s; 5.68 km / 8 min 48 s; 5.70 km / 8 min 50 s

No.	Test	Expected	Observed
4	Flood at 20.0 m with a flood source	Flooded roads cut; no route reported when A is cut off from B	4,824 intersections flooded, 5,218 segments cut; "No route"
5	Fire at 35 minutes (wind east, strength 0.40, margin 60 s)	Burning and warning zones drawn	1,968 segments burning
6	Fire at 66 minutes, A inside the burnt area	No route; reason displayed	6,597 segments burning; "No route. A is already inside the fire zone (or inside the safety margin)"
7	Evacuation under fire at 35 minutes, default capacities	Flow plan never exceeds capacity	0 people over capacity (flow) against 33,904 (nearest shelter); 12,600 of 47,636 placed
8	Evacuation with raised shelter capacities	Flow plan never exceeds capacity	0 people over capacity against 5,888 (nearest shelter); 40,720 of 47,636 placed
9	Earthquake magnitude 6.5, fragility 0.15, one random draw and Monte Carlo (100 trials)	Random damage drawn; probabilities reported	Draw damages 1,729 of 15,402 segments (11.2%); average 1,721; A and B connected 0%; 27.3% of people cut off; 85 ms
10	Services: hospitals within 10 minutes	Catchment drawn; coverage reported	62 hospitals, 100% of the population and 0 of 40 zones outside the catchment; 6.7 ms
11	Network: find bridges and cut points	Bridges and articulation points drawn	5,108 bridge segments (1,318 cutting off 10+ nodes), 4,672 articulation points; 6.2 ms
12	Network: critical roads, 80 sample sources	Ranked list of roads	Tambaram-Mudichur-Sriperumbudur Road ranked first; 320 ms
13	Earthquake: Monte Carlo with a planned route, magnitude 6.5, 100 trials	Route survival and connectivity reported	Route survives 0%, A and B connected 49%, 30.7% of people cut off; 108 ms
14	Evacuation after the earthquake, raised capacities	Flow plan never exceeds capacity	0 people over capacity against 3,552 (nearest shelter); 26,036 of 47,636 placed
15	Trip with 6 stops	Held-Karp not longer than the heuristics	Nearest neighbour 9 min 4 s; 2-opt and Held-Karp 7 min 59 s
16	Benchmark on 30 random pairs	Table of average time and nodes settled	Dijkstra 8,360 nodes; A* 5,196; bidirectional 2,881; Bellman-Ford 13,654 (8 pairs)

5.2 Application Outputs

Shortest-path comparison on the real network. For one origin–destination pair (best of three runs) the four algorithms gave the same optimal cost but did very different amounts of work:

Table 5.3: Shortest-path algorithms on the real Tambaram network (one pair, 5.68 km route)

Algorithm	Time (ms)	Nodes settled	Cost
A*	1.50	4,579	8 min 48 s
Dijkstra	1.50	7,760	8 min 48 s

Algorithm	Time (ms)	Nodes settled	Cost
Bidirectional Dijkstra	0.90	3,893	8 min 48 s
Bellman-Ford	23.30	13,645	8 min 48 s

The node counts are deterministic; the times are small and vary slightly between runs.

Average over 30 random pairs on the real network. The Benchmark button runs the algorithms on random origin-destination pairs of the real Tambaram graph (Figure 4.17):

Table 5.4: Average search effort over 30 random routes (real Tambaram network)

Algorithm	Avg. time (ms)	Avg. nodes settled	Time relative to Dijkstra
Dijkstra	1.50	8,360	1.00x
A* (haversine heuristic)	2.23	5,196	0.67x
Bidirectional Dijkstra	0.71	2,881	2.12x
Bellman-Ford (8 pairs only)	31.79	13,654	0.05x

Bidirectional Dijkstra settled 66% fewer nodes than Dijkstra and was about twice as fast, and A* settled 38% fewer nodes. Bellman-Ford is run on only 8 pairs because it is O(VE).

Average over many routes on the test graph. On the deterministic test graph (28,232 intersections), the average of 120 random drive routes was:

Table 5.5: Average search effort over 120 random routes (test graph)

Algorithm	Avg. time (ms)	Avg. nodes settled
Dijkstra	2.65	14,220
A*	1.42	5,132
Bidirectional Dijkstra	1.84	8,791
Bellman-Ford	70.48	28,225

Evacuation plan on the real network (fire 35 minutes after ignition, 40 zones, 47,636 people). The default shelter capacities (school 600, college 1,200, community hall 400, other 200) give 14,600 places in total. Raised capacities (8,000, 12,000, 4,000 and 2,000) were also tried (Figures 4.8 and 4.9):

Table 5.6: Evacuation plan versus nearest-shelter assignment

Scenario and capacities	Method	Mean trip	Longest trip	People over capacity
Fire, default capacities (14,600 places)	Min-cost max-flow (12,600 placed)	1 min 51 s	8 min 2 s	0
Fire, default capacities	Nearest shelter	3 min 23 s	10 min 28 s	33,904
Fire, raised capacities	Min-cost max-flow (40,720 placed)	3 min 26 s	10 min 28 s	0
Fire, raised capacities	Nearest shelter	3 min 23 s	10 min 28 s	5,888

Scenario and capacities	Method	Mean trip	Longest trip	People over capacity
Earthquake (M 6.5), raised capacities	Min-cost max-flow (26,036 placed)	2 min 37 s	6 min 15 s	0
Earthquake (M 6.5), raised capacities	Nearest shelter	2 min 31 s	5 min 57 s	3,552

With the default capacities the flow plan places 12,600 people, so its mean trip counts only those people while the nearest-shelter mean counts all 47,636, and the two means are not directly comparable. With raised capacities the flow plan places 40,720 people, its mean trip is almost the same as the nearest-shelter mean (3 min 26 s against 3 min 23 s) and no shelter is overfilled, whereas the nearest-shelter method overfills shelters by 5,888 people. The last column is the reliable comparison in every case; the earthquake case (Figure 4.11) shows the same pattern.

Services, network, earthquake reliability and trips. Figures 4.12 to 4.16 show the remaining features. With a 10-minute limit every one of the 62 hospitals is reachable and all 40 population zones are inside the catchment; Hindu Mission Hospital is the nearest hospital for the most people (9,800). Tarjan's algorithm found 5,108 bridge segments, so a large part of the road graph is tree-like and depends on single roads, and Brandes' algorithm ranks Tambaram-Mudichur-Sriperumbudur Road as the most heavily used road. In the Monte Carlo run (magnitude 6.5) the planned route survived in none of the 100 trials, yet origin and destination remained connected in 49% of them, which shows the value of alternative routes. For a six-stop trip the nearest-neighbour tour was 13.5% longer than the optimal tour found by 2-opt and Held-Karp.

5.3 Results and Discussion

- **Correctness.** Dijkstra, A*, bidirectional Dijkstra and Bellman-Ford returned the same optimal cost in every automated test and on every real-data comparison. This confirms that the A* heuristic is admissible and that the bidirectional stopping rule is correct.
- **Efficiency.** For the 5.68 km route A* settled 4,579 nodes against 7,760 for Dijkstra (about 41% fewer) and bidirectional search 3,893 (about 50% fewer). Averaged over 30 random routes on the real network (Table 5.4), A* settled 38% fewer nodes and bidirectional search 66% fewer. Bidirectional search was also the fastest in time (0.71 ms against 1.50 ms for Dijkstra on average), whereas A* was slower than Dijkstra in milliseconds (2.23 ms) even though it settled fewer nodes, most likely because it evaluates a haversine distance for every node it touches and the graph is small. On the larger test graph A* was fastest (Table 5.5). Bellman-Ford settled 13,645 nodes and took about 23 ms on the single pair, roughly 15 times longer than Dijkstra, which matches its O(VE) complexity.
- **Scenarios work through the graph only.** The same four algorithms handle flood, fire and earthquake without any change. The fire scenario shows the limit of this idea:

deadlines depend on arrival time, so bidirectional search cannot be used and the application falls back to A*.

- **Evacuation.** Min-cost max-flow never overfilled a shelter. With the default capacities (14,600 places for 47,636 people) it placed 12,600 people and left the rest unplaced, whereas the naive nearest-shelter method overfilled shelters by 33,904 people. With raised capacities it placed 40,720 people with no overflow at almost the same mean trip time as the nearest-shelter method, which overfilled shelters by 5,888 people. The flow model maximises the number placed first and minimises travel time second, so it does not give priority to people close to the fire; this is a limitation of the model.
- **Heuristics for trips.** In the automated tests nearest neighbour was on average 27.3% longer than the optimal tour and 2-opt 17.3% longer, which shows the value of the exact Held-Karp algorithm for small numbers of stops.
- **Alternative routes.** Yen's algorithm can return alternatives that differ from the best route by only a few segments (Figure 4.4); this is correct behaviour for k-shortest paths and is most useful when the two points are separated by a river or flyover.
- **Randomness.** The earthquake damage shown on the map is one random draw. Monte Carlo (100 draws by default) turns it into probabilities. At magnitude 6.5 and fragility 0.15 one draw damaged 11.2% of the segments (1,729 of 15,402) and the 100-trial average was 1,721. In the run of Figure 4.15 the planned route survived in none of the 100 trials, yet its end points stayed connected in 49% of them; in the run of Figure 4.10 (points farther apart) A and B were never connected, which shows why the result depends on where the two points are.

Chapter 6 – Conclusion and Future Scope

6.1 Conclusion

The project delivers a working GIS-style application for the real Tambaram road network that combines everyday routing with flood, fire and earthquake planning. Ten graph and optimisation algorithms were implemented from scratch, verified against brute-force solutions by 38 automated tests, and demonstrated on 14,022 intersections and 15,402 road segments of real data with response times of a few milliseconds per route.

The design principle that scenarios change only the graph kept the algorithms independent of the hazards and made it possible to compare them fairly. The results show the expected behaviour: A* and bidirectional search reduce the work of Dijkstra, Bellman-Ford is far slower, min-cost flow respects shelter capacities where the naive method does not, and Tarjan and betweenness expose the fragile parts of the network.

6.2 Limitations

- Road speeds by class, rush-hour multipliers, walking speed and shelter capacities are assumed values, not measurements.
- Population is estimated from building counts times people per building; there is no census data.
- Elevation is SRTM 30 m sampled on a coarse grid and interpolated, so flood boundaries are indicative only. The flood model is a water-level model without rainfall or drainage.
- The fire model spreads along streets only and the earthquake model is a simple probability law; both are explainable demonstrations, not engineering predictions.
- The evacuation plan is a snapshot: everyone starts at the same time, roads have no capacity limit, and the flow plan does not give priority to people nearest the hazard.
- Only the largest connected part of the road network is used, and the data reflect OpenStreetMap at the time of download.
- The interface was tested on a desktop browser; it has not been optimised for phones.

6.3 Future Enhancements

- Time-stepped evacuation with road capacity and congestion (time-expanded flow network).
- Risk-aware routing that penalises roads in high-risk areas, not only roads that are already blocked.
- Real traffic data, real census data and a higher-resolution elevation model.
- Rainfall-driven flood simulation and a landslide or building-collapse model.
- Contraction hierarchies or ALT landmarks for much larger areas; running heavy analyses in a web worker.

- Saving and sharing scenarios, exporting results as CSV or GeoJSON, and a mobile-friendly layout.

References

[1] OpenStreetMap contributors. OpenStreetMap data, licensed under the Open Database Licence. https://www.openstreetmap.org/copyright

[2] Overpass API documentation. https://wiki.openstreetmap.org/wiki/Overpass_API

[3] OpenTopoData. SRTM 30 m elevation API. https://www.opentopodata.org

[4] Farr, T. G. et al. (2007). The Shuttle Radar Topography Mission. Reviews of Geophysics, 45, RG2004.

[5] Leaflet 1.9.4, an open-source JavaScript library for interactive maps. https://leafletjs.com

[6] Node.js documentation. https://nodejs.org

[7] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1, 269–271.

[8] Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

[9] Pohl, I. (1971). Bi-directional search. Machine Intelligence 6, 127–140.

[10] Bellman, R. (1958). On a routing problem. Quarterly of Applied Mathematics, 16, 87–90.

[11] Yen, J. Y. (1971). Finding the k shortest loopless paths in a network. Management Science, 17(11), 712–716.

[12] Tarjan, R. (1972). Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1(2), 146–160.

[13] Brandes, U. (2001). A faster algorithm for betweenness centrality. Journal of Mathematical Sociology, 25(2), 163–177.

[14] Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. (1993). Network Flows: Theory, Algorithms, and Applications. Prentice Hall.

[15] Held, M. and Karp, R. M. (1962). A dynamic programming approach to sequencing problems. Journal of the SIAM, 10(1), 196–210.

[16] Croes, G. A. (1958). A method for solving traveling-salesman problems. Operations Research, 6(6), 791–812.

[17] Arthur, D. and Vassilvitskii, S. (2007). k-means++: the advantages of careful seeding. Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms, 1027–1035.

[18] Nemhauser, G. L., Wolsey, L. A., and Fisher, M. L. (1978). An analysis of approximations for maximizing submodular set functions I. Mathematical Programming, 14, 265–294.

[19] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms, 4th edition. MIT Press.

Appendices

Appendix A: Project Structure and How to Run

```
tambaram-geo/
  index.html              page layout (scenario panel, 5 tabs, map)
  css/style.css           styling
  js/core.js              heap, union-find, spatial grid, haversine, RNG
  js/graph.js             graph and CSR adjacency, cost arrays
  js/algos.js             all algorithms
  js/scenarios.js         flood, fire, earthquake, rush hour
  js/app.js               user interface logic
  lib/                    Leaflet 1.9.4
  data/dataset.js         generated real data (window.GEO_DATASET)
  tools/fetch-osm.js      downloads the data (Node.js, needs internet)
  tools/test.js           38 algorithm tests
  tools/test-fetch.js     data-parsing tests
```

To run: (1) `node tools/fetch-osm.js` once to download Tambaram data (a different area can be chosen with `--bbox south,west,north,east`); (2) open `index.html` in a browser (add `?tiles=none` to the address to hide the background map); (3) `node tools/test.js` to run the tests. Source code repository: [add your GitHub link here if you upload the project]

Appendix B: Assumed Model Parameters

Table B.1: Assumed speeds and rush-hour multipliers by road class

Road class	Free-flow speed (km/h)	Rush-hour multiplier
motorway	70	1.6
trunk	50	1.9
primary	40	1.9
secondary	32	1.7
tertiary	26	1.4
unclassified	22	1.2
residential	18	1.1
living street	10	1.0
other	15	1.1

Walking speed: 1.35 m/s. Wading through water up to 0.5 m deep: three times slower. Bridges are 15 times more likely to fail in the earthquake model. Default shelter capacities (editable in the interface): school 600, college 1,200, community hall 400, other 200. Default population: 4 people per building in 40 zones.

Appendix C: Key Source Code

The main algorithms and scenario models, copied unchanged from the source files in the js folder. The complete source code is available in the project repository.

```
function sssp(view, w, sources, opts) {
  opts = opts || {};
  const n = view.n, reverse = !!opts.reverse;
  const start = reverse ? view.rstart : view.start;
  const nbr = reverse ? view.rhead : view.head;
  const arcOf = view.rarc;
  const dist = new Float64Array(n).fill(Infinity);
  const prev = new Int32Array(n).fill(-1);
  const label = new Int32Array(n).fill(-1);
  const heap = new MinHeap(1024);
  const maxCost = opts.maxCost === undefined ? Infinity : opts.maxCost;
  const deadline = reverse ? null : opts.deadline, t0 = opts.t0 || 0;
  const blockedNode = opts.blockedNode, target = opts.target === undefined ? -1 :
opts.target;
  const order = opts.collect ? [] : null;
  for (let i = 0; i < sources.length; i++) {
    const s = sources[i];
    if (deadline && t0 >= deadline[s]) continue;
    if (dist[s] > 0) { dist[s] = 0; label[s] = i; heap.push(0, s); }
  }
  let settled = 0, relaxed = 0;
  while (heap.size) {
    const u = heap.pop(), du = heap.topKey;
    if (du > dist[u]) continue;
    settled++;
    if (order) order.push(u);
    if (u === target) break;
    for (let i = start[u], e = start[u + 1]; i < e; i++) {
      const a = reverse ? arcOf[i] : i;
      const c = w[a];
      if (c === Infinity) continue;
      const v = nbr[i];
      if (blockedNode && blockedNode[v]) continue;
      const nd = du + c;
      if (nd >= dist[v] || nd > maxCost) continue;
      if (deadline && t0 + nd >= deadline[v]) continue;
      dist[v] = nd; prev[v] = a; label[v] = label[u];
      heap.push(nd, v); relaxed++;
    }
  }
  return { dist, prev, label, settled, relaxed, order };
}
```

Listing C.1: Single-source / multi-source Dijkstra (js/algos.js)

```
function astar(G, view, w, s, t, hFactor, opts) {
  opts = opts || {};
  const t0clock = now();
  const n = view.n, start = view.start, head = view.head;
  const lat = G.lat, lon = G.lon, tl = lat[t], tn = lon[t];
  const deadline = opts.deadline, t0 = opts.t0 || 0, blockedNode = opts.blockedNode;
  const g = new Float64Array(n).fill(Infinity);
  const prev = new Int32Array(n).fill(-1);
```

```
  const hc = new Float64Array(n).fill(-1);
  const h = (u) => {
    let x = hc[u];
    if (x < 0) { x = haversine(lat[u], lon[u], tl, tn) * hFactor; hc[u] = x; }
    return x;
  };
  const heap = new MinHeap(1024), order = [];
  let settled = 0, relaxed = 0;
  if (!(deadline && t0 >= deadline[s])) { g[s] = 0; heap.push(h(s), s); }
  while (heap.size) {
    const u = heap.pop(), f = heap.topKey;
    if (f > g[u] + h(u) + 1e-9) continue;
    settled++; order.push(u);
    if (u === t) break;
    const gu = g[u];
    for (let i = start[u], e = start[u + 1]; i < e; i++) {
      const c = w[i];
      if (c === Infinity) continue;
      const v = head[i];
      if (blockedNode && blockedNode[v]) continue;
      const ng = gu + c;
      if (ng >= g[v]) continue;
      if (deadline && t0 + ng >= deadline[v]) continue;
      g[v] = ng; prev[v] = i;
      heap.push(ng + h(v), v); relaxed++;
    }
  }
  const found = g[t] < Infinity;
  const arcs = found ? pathFromPrev(view, prev, s, t) : null;
  return { algo: 'astar', found, cost: g[t], arcs, nodes: found ? arcsToNodes(view,
arcs, s) : null,
    settled, relaxed, ms: now() - t0clock, order };
}
```

Listing C.2: A search (js/algos.js)*

```
function bidirectional(view, w, s, t) {
  const t0clock = now();
  const n = view.n;
  const distF = new Float64Array(n).fill(Infinity), distB = new
Float64Array(n).fill(Infinity);
  const prevF = new Int32Array(n).fill(-1), prevB = new Int32Array(n).fill(-1);
  const hf = new MinHeap(1024), hb = new MinHeap(1024);
  const orderF = [], orderB = [];
  distF[s] = 0; distB[t] = 0; hf.push(0, s); hb.push(0, t);
  let mu = Infinity, meet = -1, relaxed = 0;
  if (s === t) { mu = 0; meet = s; }
  const clean = (h, d) => { while (h.size && h.k[0] > d[h.v[0]]) h.pop(); };
  while (hf.size && hb.size && mu > 0) {
    clean(hf, distF); clean(hb, distB);
    if (!hf.size || !hb.size) break;
    if (hf.k[0] + hb.k[0] >= mu) break;
    if (hf.size <= hb.size) {
      const u = hf.pop(), du = hf.topKey;
      if (du > distF[u]) continue;
      orderF.push(u);
      for (let i = view.start[u], e = view.start[u + 1]; i < e; i++) {
        const c = w[i]; if (c === Infinity) continue;
```

```
        const v = view.head[i], nd = du + c;
        if (nd < distF[v]) {
          distF[v] = nd; prevF[v] = i; hf.push(nd, v); relaxed++;
          if (distB[v] < Infinity && nd + distB[v] < mu) { mu = nd + distB[v]; meet = v;
}
        }
      }
    } else {
      const u = hb.pop(), du = hb.topKey;
      if (du > distB[u]) continue;
      orderB.push(u);
      for (let i = view.rstart[u], e = view.rstart[u + 1]; i < e; i++) {
        const a = view.rarc[i], c = w[a]; if (c === Infinity) continue;
        const v = view.rhead[i], nd = du + c;
        if (nd < distB[v]) {
          distB[v] = nd; prevB[v] = a; hb.push(nd, v); relaxed++;
          if (distF[v] < Infinity && nd + distF[v] < mu) { mu = nd + distF[v]; meet = v;
}
        }
      }
    }
  }
  const found = meet >= 0 && mu < Infinity;
  let arcs = null;
  if (found) {
    arcs = pathFromPrev(view, prevF, s, meet) || [];
    let v = meet;
    while (v !== t) { const a = prevB[v]; arcs.push(a); v = view.head[a]; }
  }
  return { algo: 'bidir', found, cost: found ? mu : Infinity, arcs, nodes: found ?
arcsToNodes(view, arcs, s) : null,
    settled: orderF.length + orderB.length, relaxed, ms: now() - t0clock, orderF, orderB
};
}
```

Listing C.3: Bidirectional Dijkstra (js/algos.js)

```
function yen(G, view, w, s, t, K, metric, opts) {
  opts = opts || {};
  const t0clock = now();
  const hF = view.hFactor[metric === 'distance' ? 'distance' : 'time'];
  const baseT0 = opts.t0 || 0;
  const first = astar(G, view, w, s, t, hF, opts);
  if (!first.found) return { paths: [], ms: now() - t0clock, spurs: 0 };
  const A = [{ arcs: first.arcs, cost: first.cost }], B = [];
  const seen = new Set([first.arcs.join(',')]);
  const blockedNode = new Uint8Array(view.n);
  let spurs = 0;
  for (let k = 1; k < K; k++) {
    const pa = A[k - 1].arcs;
    let rootCost = 0;
    for (let i = 0; i < pa.length; i++) {
      const spurNode = i === 0 ? s : view.head[pa[i - 1]];
      if (i > 0) rootCost += w[pa[i - 1]];
      const removed = [];
      for (const P of A) {
        if (P.arcs.length > i) {
          let same = true;
```

```
          for (let j = 0; j < i; j++) if (P.arcs[j] !== pa[j]) { same = false; break; }
          if (same) { const a = P.arcs[i]; if (w[a] !== Infinity) { removed.push(a,
w[a]); w[a] = Infinity; } }
        }
      }
      const rootNodes = [];
      let v = s;
      for (let j = 0; j < i; j++) { blockedNode[v] = 1; rootNodes.push(v); v =
view.head[pa[j]]; }
      let hasOut = false;
      for (let q = view.start[spurNode]; q < view.start[spurNode + 1]; q++) {
        if (w[q] !== Infinity && !blockedNode[view.head[q]]) { hasOut = true; break; }
      }
      if (hasOut) {
        spurs++;
        const sp = astar(G, view, w, spurNode, t, hF, Object.assign({}, opts,
{ blockedNode, t0: baseT0 + rootCost }));
        if (sp.found) {
          const arcs = pa.slice(0, i).concat(sp.arcs), key = arcs.join(',');
          if (!seen.has(key)) { seen.add(key); B.push({ arcs, cost: rootCost +
sp.cost }); }
        }
      }
      for (let q = 0; q < removed.length; q += 2) w[removed[q]] = removed[q + 1];
      for (const x of rootNodes) blockedNode[x] = 0;
    }
    if (!B.length) break;
    let bi = 0;
    for (let j = 1; j < B.length; j++) if (B[j].cost < B[bi].cost) bi = j;
    A.push(B[bi]); B.splice(bi, 1);
  }
  return { paths: A.map(p => ({ arcs: p.arcs, cost: p.cost, nodes: arcsToNodes(view,
p.arcs, s) })), ms: now() - t0clock, spurs };
}
```

Listing C.4: Yen's k shortest loop-free paths (js/algos.js)

```
function tarjan(view, blockedSeg) {
  const n = view.n, start = view.start, head = view.head, seg = view.seg;
  const disc = new Int32Array(n).fill(-1), low = new Int32Array(n), sz = new
Int32Array(n);
  const parent = new Int32Array(n).fill(-1), parSeg = new Int32Array(n).fill(-1), rootOf
= new Int32Array(n);
  const ptr = new Int32Array(n), stack = new Int32Array(n);
  const art = new Uint8Array(n), cut = new Int32Array(n), rootKids = new Int32Array(n);
  const bridges = [];
  let timer = 0;
  for (let r = 0; r < n; r++) {
    if (disc[r] !== -1) continue;
    let sp = 0;
    stack[sp++] = r; disc[r] = low[r] = timer++; sz[r] = 1; rootOf[r] = r; ptr[r] =
start[r];
    while (sp > 0) {
      const u = stack[sp - 1];
      if (ptr[u] < start[u + 1]) {
        const a = ptr[u]++;
        const sg = seg[a];
        if (blockedSeg && blockedSeg[sg]) continue;
```

```
        if (sg === parSeg[u]) continue;
        const v = head[a];
        if (disc[v] === -1) {
          disc[v] = low[v] = timer++; sz[v] = 1; parent[v] = u; parSeg[v] = sg;
rootOf[v] = r; ptr[v] = start[v];
          stack[sp++] = v;
        } else if (disc[v] < low[u]) low[u] = disc[v];
      } else {
        sp--;
        const p = parent[u];
        if (p >= 0) {
          if (low[u] < low[p]) low[p] = low[u];
          sz[p] += sz[u];
          if (low[u] > disc[p]) bridges.push({ seg: parSeg[u], child: u, below: sz[u],
root: rootOf[u] });
          if (p !== r) { if (low[u] >= disc[p]) { art[p] = 1; if (sz[u] > cut[p]) cut[p]
= sz[u]; } }
          else rootKids[r]++;
        }
      }
    }
    if (rootKids[r] >= 2) { art[r] = 1; cut[r] = Math.max(cut[r], 1); }
  }
  // Component size lets us report the *smaller* side that a bridge isolates.
  const out = bridges.map(b => {
    const comp = sz[b.root];
    return { seg: b.seg, isolated: Math.min(b.below, comp - b.below), component: comp };
  });
  out.sort((x, y) => y.isolated - x.isolated);
  const artNodes = [];
  for (let v = 0; v < n; v++) if (art[v]) artNodes.push(v);
  return { bridges: out, articulation: artNodes };
}
```

Listing C.5: Tarjan bridges and articulation points, iterative (js/algos.js)

```
function edgeBetweenness(view, w, m, k, seed, sources) {
  const n = view.n, rng = mulberry32(seed || 7);
  const bc = new Float64Array(m);
  const dist = new Float64Array(n), sigma = new Float64Array(n), delta = new
Float64Array(n);
  const order = new Int32Array(n), done = new Uint8Array(n);
  const heap = new MinHeap(1024);
  const eps = (d) => 1e-9 * (1 + d);
  for (let it = 0; it < k; it++) {
    const s = sources ? sources[it] : Math.floor(rng() * n);
    dist.fill(Infinity); sigma.fill(0); delta.fill(0); done.fill(0);
    dist[s] = 0; sigma[s] = 1; heap.size = 0; heap.push(0, s);
    let cnt = 0;
    while (heap.size) {
      const u = heap.pop(), du = heap.topKey;
      if (done[u] || du > dist[u]) continue;
      done[u] = 1; order[cnt++] = u;
      for (let a = view.start[u], e = view.start[u + 1]; a < e; a++) {
        const c = w[a]; if (c === Infinity) continue;
        const v = view.head[a], nd = du + c;
        if (nd < dist[v]) { dist[v] = nd; heap.push(nd, v); }
      }
```

```
    }
    for (let i = 0; i < cnt; i++) {
      const u = order[i];
      for (let a = view.start[u], e = view.start[u + 1]; a < e; a++) {
        const c = w[a]; if (c === Infinity) continue;
        const v = view.head[a];
        if (Math.abs(dist[u] + c - dist[v]) <= eps(dist[v])) sigma[v] += sigma[u];
      }
    }
    for (let i = cnt - 1; i >= 0; i--) {
      const x = order[i];
      for (let q = view.rstart[x], e = view.rstart[x + 1]; q < e; q++) {
        const a = view.rarc[q], c = w[a]; if (c === Infinity) continue;
        const v = view.rhead[q];
        if (dist[v] === Infinity) continue;
        if (Math.abs(dist[v] + c - dist[x]) <= eps(dist[x])) {
          const coef = (sigma[v] / sigma[x]) * (1 + delta[x]);
          delta[v] += coef; bc[view.seg[a]] += coef;
        }
      }
    }
  }
  return bc;
}
```

Listing C.6: Brandes edge betweenness (js/algos.js)

```
function minCostMaxFlow(N, edges, s, t, limit) {
  const to = [], cap = [], cost = [], nxt = [];
  const head = new Int32Array(N).fill(-1);
  const add = (u, v, c, w) => {
    to.push(v); cap.push(c); cost.push(w); nxt.push(head[u]); head[u] = to.length - 1;
    to.push(u); cap.push(0); cost.push(-w); nxt.push(head[v]); head[v] = to.length - 1;
  };
  for (const e of edges) add(e.u, e.v, e.cap, e.cost);
  let flow = 0, total = 0;
  limit = limit === undefined ? Infinity : limit;
  const dist = new Float64Array(N), inq = new Uint8Array(N), pe = new Int32Array(N);
  let augmentations = 0;
  while (flow < limit) {
    dist.fill(Infinity); pe.fill(-1); dist[s] = 0;
    const q = [s]; inq[s] = 1;
    for (let qi = 0; qi < q.length; qi++) {
      const u = q[qi]; inq[u] = 0;
      for (let e = head[u]; e !== -1; e = nxt[e]) {
        if (cap[e] > 0 && dist[u] + cost[e] < dist[to[e]] - 1e-12) {
          dist[to[e]] = dist[u] + cost[e]; pe[to[e]] = e;
          if (!inq[to[e]]) { inq[to[e]] = 1; q.push(to[e]); }
        }
      }
      if (q.length > 4 * N * N + 100) break; // safety net against numeric cycling
    }
    if (dist[t] === Infinity) break;
    let f = limit - flow;
    for (let v = t; v !== s; v = to[pe[v] ^ 1]) f = Math.min(f, cap[pe[v]]);
    for (let v = t; v !== s; v = to[pe[v] ^ 1]) { cap[pe[v]] -= f; cap[pe[v] ^ 1] +=
f; }
    flow += f; total += f * dist[t]; augmentations++;
```

```
  }
  const edgeFlow = new Float64Array(edges.length);
  for (let i = 0; i < edges.length; i++) edgeFlow[i] = cap[2 * i + 1];
  return { flow, cost: total, edgeFlow, augmentations };
}
```

Listing C.7: Min-cost max-flow by successive shortest paths (js/algos.js)

```
function kmeans(pts, k, seed, iters) {
  const n = pts.length / 2;
  k = Math.min(k, n);
  iters = iters || 20;
  const rng = mulberry32(seed || 11);
  let lat0 = 0;
  for (let i = 0; i < n; i++) lat0 += pts[2 * i];
  lat0 /= n;
  const kx = Math.cos(lat0 * Math.PI / 180);
  const px = new Float64Array(n), py = new Float64Array(n);
  for (let i = 0; i < n; i++) { py[i] = pts[2 * i]; px[i] = pts[2 * i + 1] * kx; }
  const cx = new Float64Array(k), cy = new Float64Array(k);
  const first = Math.floor(rng() * n);
  cx[0] = px[first]; cy[0] = py[first];
  const d2 = new Float64Array(n).fill(Infinity);
  for (let c = 1; c < k; c++) {
    let sum = 0;
    for (let i = 0; i < n; i++) {
      const dx = px[i] - cx[c - 1], dy = py[i] - cy[c - 1];
      const d = dx * dx + dy * dy;
      if (d < d2[i]) d2[i] = d;
      sum += d2[i];
    }
    let r = rng() * sum, pick = n - 1;
    for (let i = 0; i < n; i++) { r -= d2[i]; if (r <= 0) { pick = i; break; } }
    cx[c] = px[pick]; cy[c] = py[pick];
  }
  const assign = new Int32Array(n).fill(-1), count = new Int32Array(k);
  for (let it = 0; it < iters; it++) {
    let moved = 0;
    for (let i = 0; i < n; i++) {
      let best = 0, bd = Infinity;
      for (let c = 0; c < k; c++) {
        const dx = px[i] - cx[c], dy = py[i] - cy[c], d = dx * dx + dy * dy;
        if (d < bd) { bd = d; best = c; }
      }
      if (assign[i] !== best) { assign[i] = best; moved++; }
    }
    const sx = new Float64Array(k), sy = new Float64Array(k);
    count.fill(0);
    for (let i = 0; i < n; i++) { sx[assign[i]] += px[i]; sy[assign[i]] += py[i];
count[assign[i]]++; }
    for (let c = 0; c < k; c++) if (count[c] > 0) { cx[c] = sx[c] / count[c]; cy[c] =
sy[c] / count[c]; }
    if (moved === 0) break;
  }
  const cent = [];
  for (let c = 0; c < k; c++) if (count[c] > 0) cent.push({ lat: cy[c], lon: cx[c] / kx,
count: count[c] });
  return cent;
```

```
}
```

Listing C.8: k-means++ clustering for population zones (js/algos.js)

```
function greedyCoverage(pop, covers, k, preCovered) {
  const Z = pop.length;
  const covered = preCovered ? Uint8Array.from(preCovered) : new Uint8Array(Z);
  let base = 0, total = 0;
  for (let z = 0; z < Z; z++) { total += pop[z]; if (covered[z]) base += pop[z]; }
  const chosen = [], gains = [];
  const taken = new Uint8Array(covers.length);
  let acc = base;
  for (let step = 0; step < k; step++) {
    let best = -1, bestGain = 0;
    for (let c = 0; c < covers.length; c++) {
      if (taken[c]) continue;
      let g = 0;
      const zs = covers[c];
      for (let i = 0; i < zs.length; i++) if (!covered[zs[i]]) g += pop[zs[i]];
      if (g > bestGain) { bestGain = g; best = c; }
    }
    if (best < 0) break;
    taken[best] = 1;
    for (const z of covers[best]) covered[z] = 1;
    acc += bestGain;
    chosen.push(best); gains.push({ gain: bestGain, coverage: acc / total });
  }
  return { chosen, gains, baseCoverage: total ? base / total : 0, total };
}
```

Listing C.9: Greedy maximum coverage for shelter placement (js/algos.js)

```
function tsp2opt(D, tour) {
  tour = tour.slice();
  let improved = true, cost = tourCost(D, tour);
  while (improved) {
    improved = false;
    for (let i = 1; i < tour.length - 1; i++) {
      for (let j = i + 1; j < tour.length; j++) {
        const cand = tour.slice(0, i).concat(tour.slice(i, j + 1).reverse(),
tour.slice(j + 1));
        const c = tourCost(D, cand);
        if (c < cost - 1e-9) { tour = cand; cost = c; improved = true; }
      }
    }
  }
  return tour;
}
```

Listing C.10: 2-opt improvement for trips (js/algos.js)

```
function tspHeldKarp(D) {
  const n = D.length;
  if (n <= 1) return [0];
  const m = n - 1, FULL = 1 << m;
  const dp = new Float64Array(FULL * m).fill(Infinity), par = new Int8Array(FULL *
m).fill(-1);
  for (let j = 0; j < m; j++) dp[(1 << j) * m + j] = D[0][j + 1];
  for (let mask = 1; mask < FULL; mask++) {
    for (let j = 0; j < m; j++) {
```

```
      if (!(mask & (1 << j))) continue;
      const cur = dp[mask * m + j];
      if (cur === Infinity) continue;
      for (let k = 0; k < m; k++) {
        if (mask & (1 << k)) continue;
        const nm = mask | (1 << k), val = cur + D[j + 1][k + 1];
        if (val < dp[nm * m + k]) { dp[nm * m + k] = val; par[nm * m + k] = j; }
      }
    }
  }
  let best = Infinity, bj = -1;
  for (let j = 0; j < m; j++) {
    const val = dp[(FULL - 1) * m + j] + D[j + 1][0];
    if (val < best) { best = val; bj = j; }
  }
  if (bj < 0) return null;
  const tour = [];
  let mask = FULL - 1, j = bj;
  while (j >= 0) { tour.push(j + 1); const pj = par[mask * m + j]; mask ^= (1 << j); j =
pj; }
  tour.push(0);
  return tour.reverse();
}
```

Listing C.11: Held-Karp exact trip planning (js/algos.js)

```
function buildGraph(ds) {
  const rawN = ds.nodes.length, ways = ds.ways;
  let total = 0;
  for (const w of ways) total += Math.max(0, w.n.length - 1);
  const rA = new Int32Array(total), rB = new Int32Array(total), rW = new
Int32Array(total);
  let k = 0;
  for (let wi = 0; wi < ways.length; wi++) {
    const nn = ways[wi].n;
    for (let j = 0; j < nn.length - 1; j++) {
      if (nn[j] === nn[j + 1]) continue;
      rA[k] = nn[j]; rB[k] = nn[j + 1]; rW[k] = wi; k++;
    }
  }
  const m0 = k;
  const uf = new UnionFind(rawN), used = new Uint8Array(rawN);
  for (let i = 0; i < m0; i++) { uf.union(rA[i], rB[i]); used[rA[i]] = 1; used[rB[i]] =
1; }
  const comp = new Int32Array(rawN);
  for (let v = 0; v < rawN; v++) if (used[v]) comp[uf.find(v)]++;
  let best = -1, bestSize = 0;
  for (let v = 0; v < rawN; v++) if (comp[v] > bestSize) { bestSize = comp[v]; best = v;
}

  const map = new Int32Array(rawN).fill(-1);
  let n = 0;
  for (let v = 0; v < rawN; v++) if (used[v] && uf.find(v) === best) map[v] = n++;
  const lat = new Float64Array(n), lon = new Float64Array(n);
  for (let v = 0; v < rawN; v++) if (map[v] >= 0) { lat[map[v]] = ds.nodes[v][0];
lon[map[v]] = ds.nodes[v][1]; }

  let m = 0;
```

```
  for (let i = 0; i < m0; i++) if (map[rA[i]] >= 0) m++;
  const segA = new Int32Array(m), segB = new Int32Array(m), segWay = new Int32Array(m);
  const segLen = new Float64Array(m), segClass = new Uint8Array(m);
  const segOneway = new Int8Array(m), segBridge = new Uint8Array(m);
  let q = 0;
  for (let i = 0; i < m0; i++) {
    if (map[rA[i]] < 0) continue;
    const a = map[rA[i]], b = map[rB[i]], w = ways[rW[i]];
    segA[q] = a; segB[q] = b; segWay[q] = rW[i];
    segLen[q] = Math.max(0.5, haversine(lat[a], lon[a], lat[b], lon[b]));
    segClass[q] = classIndex(w.h);
    segOneway[q] = w.o || 0;
    segBridge[q] = w.b ? 1 : 0;
    q++;
  }
  const G = {
    n, m, lat, lon, segA, segB, segLen, segClass, segOneway, segBridge, segWay,
    wayName: ways.map(w => w.nm || ''), wayClass: ways.map(w => classIndex(w.h)),
    droppedNodes: rawN - n
  };
  G.elev = new Float64Array(n).fill(NaN);
  G.hasElev = false; G.elevMin = NaN; G.elevMax = NaN;
  if (ds.elev && ds.elev.v && ds.elev.v.length) {
    let lo = Infinity, hi = -Infinity;
    for (let i = 0; i < n; i++) {
      const e = sampleElevation(ds.elev, lat[i], lon[i]);
      G.elev[i] = e; if (e < lo) lo = e; if (e > hi) hi = e;
    }
    G.hasElev = true; G.elevMin = lo; G.elevMax = hi;
  }
  G.grid = new SpatialGrid(lat, lon, 0.003);
  return G;
}
```

Listing C.12: Building the road graph from the dataset (js/graph.js)

```
function flood(G, walkView, p) {
  const n = G.n, flooded = new Uint8Array(n);
  let count = 0;
  if (p.src !== null && p.src !== undefined && p.connected) {
    const q = [];
    if (G.elev[p.src] <= p.level) { flooded[p.src] = 1; q.push(p.src); }
    for (let qi = 0; qi < q.length; qi++) {
      const u = q[qi];
      for (let a = walkView.start[u]; a < walkView.start[u + 1]; a++) {
        const v = walkView.head[a];
        if (!flooded[v] && G.elev[v] <= p.level) { flooded[v] = 1; q.push(v); }
      }
    }
  } else {
    for (let i = 0; i < n; i++) if (G.elev[i] <= p.level) flooded[i] = 1;
  }
  const floodedSeg = new Uint8Array(G.m);
  let segs = 0;
  for (let s = 0; s < G.m; s++) {
    // A road is impassable if either end is under water; bridges stay open.
    if ((flooded[G.segA[s]] || flooded[G.segB[s]]) && !G.segBridge[s]) { floodedSeg[s] =
1; segs++; }
```

```
  }
  for (let i = 0; i < n; i++) count += flooded[i];
  return { flooded, floodedSeg, nodes: count, segs };
}
```

Listing C.13: Flood model (js/scenarios.js)

```
function fireArrival(G, walkView, p) {
  const A = walkView.A, w = new Float64Array(A);
  const speed = p.speedMPerMin / 60;
  for (let a = 0; a < A; a++) {
    const u = walkView.from[a], v = walkView.head[a];
    const br = bearing(G.lat[u], G.lon[u], G.lat[v], G.lon[v]);
    const rate = speed * (1 + p.windK * Math.cos((br - p.windDeg) * Math.PI / 180));
    w[a] = walkView.len[a] / Math.max(rate, speed * 0.1);
  }
  const r = algos.sssp(walkView, w, p.ignitions, {});
  return r.dist;
}
```

Listing C.14: Fire arrival times with wind (js/scenarios.js)

```
function quakeProb(G, p) {
  const out = new Float64Array(G.m);
  for (let s = 0; s < G.m; s++) {
    const a = G.segA[s], b = G.segB[s];
    const la = (G.lat[a] + G.lat[b]) / 2, lo = (G.lon[a] + G.lon[b]) / 2;
    const dKm = haversine(p.epiLat, p.epiLon, la, lo) / 1000;
    const intensity = p.mag - 3.2 * Math.log10(dKm + 0.5) - 4.6;
    const base = 1 / (1 + Math.exp(-1.5 * intensity));
    const mult = G.segBridge[s] ? p.bridgeMult : 1;
    out[s] = Math.min(0.95, base * p.fragility * mult);
  }
  return out;
}
```

Listing C.15: Earthquake segment failure probability (js/scenarios.js)

```
function quakeMonteCarlo(G, prob, trials, seed, q) {
  const rng = mulberry32(seed || 1);
  const zoneNodes = q.zoneNodes || [], safeNodes = q.safeNodes || [];
  const isolated = new Float64Array(zoneNodes.length);
  let connected = 0, intact = 0, failedTotal = 0;
  for (let t = 0; t < trials; t++) {
    const f = sampleFailures(prob, rng);
    failedTotal += f.count;
    const uf = new UnionFind(G.n);
    for (let s = 0; s < G.m; s++) if (!f.failed[s]) uf.union(G.segA[s], G.segB[s]);
    if (q.pair && uf.find(q.pair[0]) === uf.find(q.pair[1])) connected++;
    if (q.routeSegs) {
      let ok = true;
      for (let i = 0; i < q.routeSegs.length; i++) if (f.failed[q.routeSegs[i]]) { ok =
false; break; }
      if (ok) intact++;
    }
    if (zoneNodes.length) {
      const safeRoots = new Set();
      for (const sn of safeNodes) safeRoots.add(uf.find(sn));
      for (let z = 0; z < zoneNodes.length; z++) if (!
safeRoots.has(uf.find(zoneNodes[z]))) isolated[z]++;
```

```
    }
  }
  for (let z = 0; z < isolated.length; z++) isolated[z] /= trials;
  return {
    trials, pConnected: q.pair ? connected / trials : null, pRouteIntact: q.routeSegs ?
intact / trials : null,
    zoneIsolated: isolated, meanFailedSegs: failedTotal / trials
  };
}
```

Listing C.16: Monte Carlo earthquake reliability (js/scenarios.js)